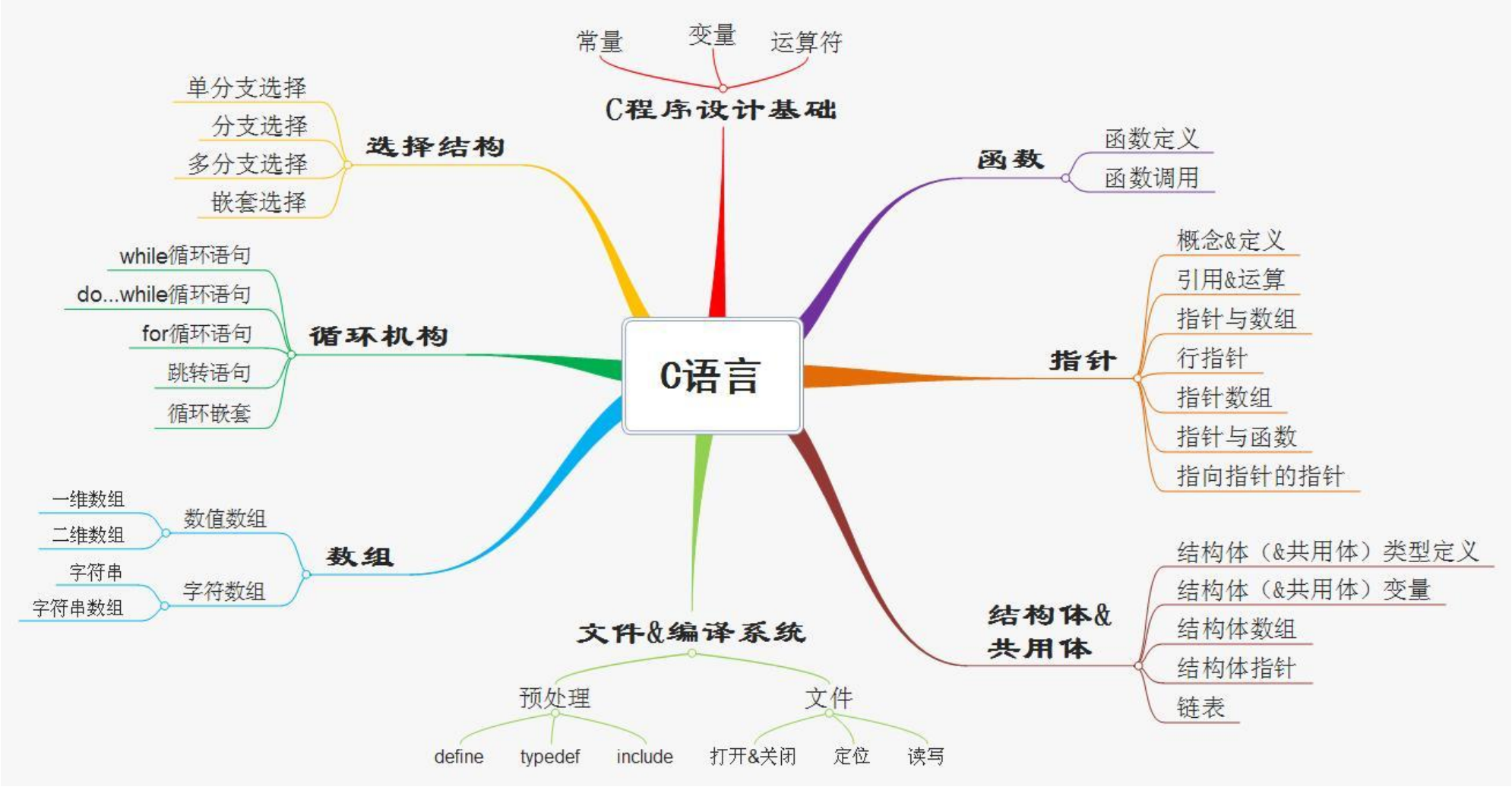
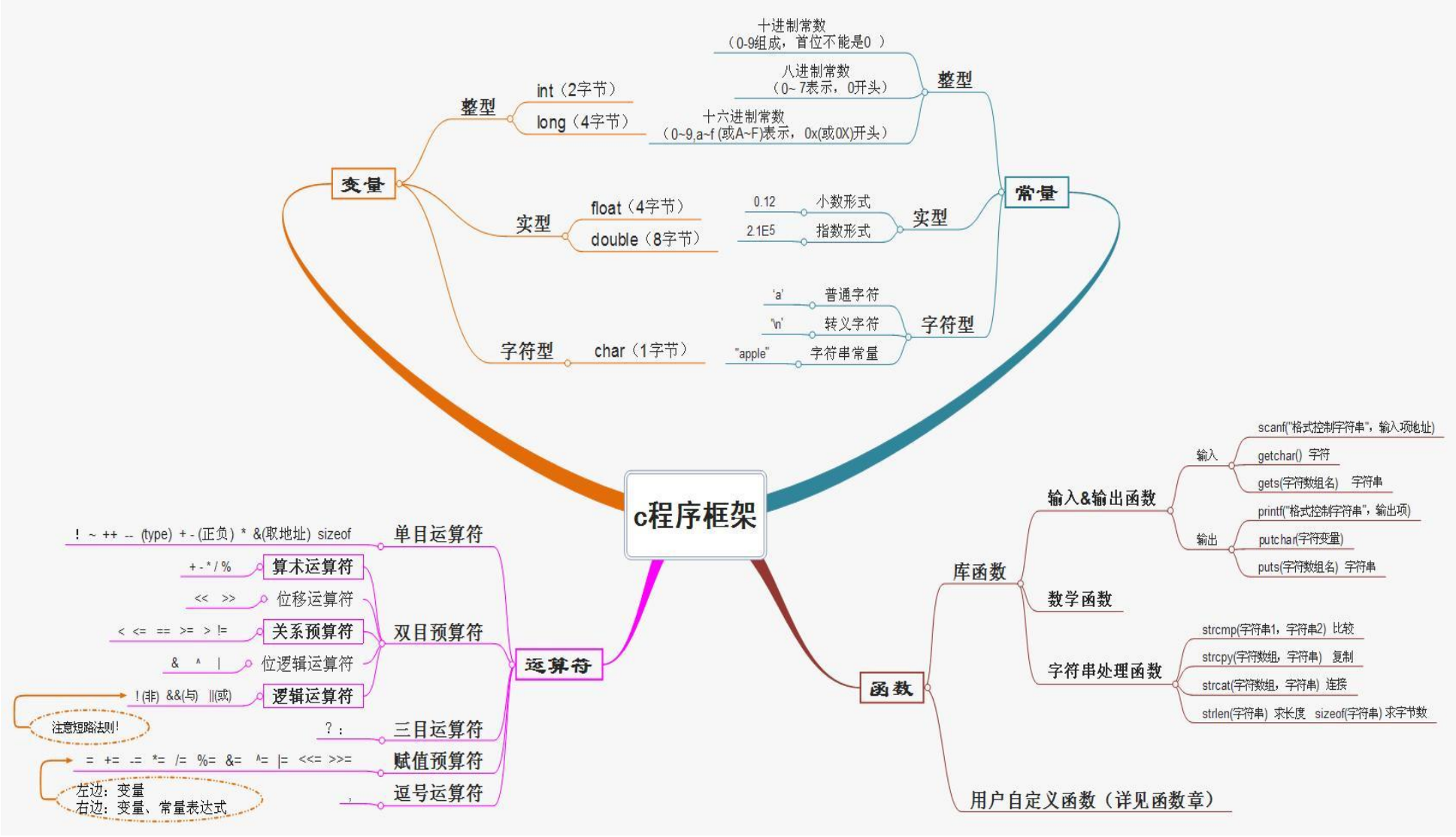
C程序设计知识点总结



思维导图 1：C 语言程序设计知识结构

第一课 C 语言程序设计基础



C程序设计知识点总结

思维导图 2：第一课 C 语言程序设计基础

本课主要知识点：

1.知识点：C 程序基础

- C 语言是一种结构化程序设计语言。三种基本结构：**顺序、选择、循环**。

例 1：（2010-09-11）以下关于结构化程序设计的叙述中正确的是（C）

A） 一个结构化程序必须同时由顺序、分支、循环三种结构组成

B）结构化程序使用 goto 语句会很便捷

C）在 C 语言中，程序的模块化是利用函数实现的

D） 由三种基本结构构成的程序只能解决小规模的问题

例 2：（2011-03-26）计算机高级语言程序的运行方法有编译执行和解释执行两种，以下叙述中正确的是（A）

A)C 语言程序仅可以编译执行

B)C 语言程序仅可以解释执行

C)C 语言程序既可以编译执行又可以解释执行

D)以上说法都不对

【解析】计算机不能直接理解高级语言，只能直接理解机器语言，所以必须要把高级语言翻译成机器语言，计算机才能执行高级语言编写的程序。 翻译的方式有两种，一个是编译，一个是解释。C 语言程序仅可以编译。

例 3：（2011-03-26）以下叙述中错误的是(D)

A)C 语言的可执行程序是由一系列机器指令构成的

B)用 C 语言编写的源程序不能直接在计算机上运行

C)通过编译得到的二进制目标程序需要连接才可以运行

D)在没有安装 C 语言集成开发环境的机器上不能运行 C 源程序生成的.exe 文件

【解析】C 程序开发过程：编辑—>（.c）编译—>（.obj）链接—>可执行文件（.exe）

2.知识点：C 语言程序的构成

- **C 程序是由函数构成的，一个 C 程序有且仅有一个 main 函数。**一个函数由两部分组成：函数的首部和函数体。函数体包括声明部分和执行部分。**一个 C 程序总是从 main 函数开始执行（即程序的入口），由 main 函数结束，（即程序的出口）。**

例 1：（2010-03-12）以下叙述正确的是（B）

A） C 语言程序是由过程和函数组成的

B）C 语言函数可以嵌套调用，例如：fun(fun(x))

C）C 语言函数不可以单独编译

D） C 语言中除了 main 函数，其他函数不可作为单独文件形式存在

例 2：（2011-09-11）以下叙述中错误的是 **（C）**

A.C 语言编写的函数源程序，其文件名后缀可以是 C

B.C 语言编写的函数都可以作为一个独立的源程序文件

C.C 语言编写的每个函数都可以进行独立的编译并执行

D.一个 C 语言程序只能有一个主函数

3.知识点：标识符

- **命名规则：**

①标识符中只能包含字母（a-z,A-Z），数字(0-9),下划线(_)。

②第一个字符必须是字母或下划线。

- 分类

都是小写
- ①关键字（32 个）（有固定含义，不可改变）
  - 类型说明符：int,float,double 等
  - 语句定义符：if，for,while 等
- ②预定义标识符（有特殊含义，可以改变）
  - 库函数的名字：printf，scanf 等
  - 编译预处理命令名：include 等

③用户自定义的标识符 i 格式合法，ii 不能使用关键字，iii 见名知意

例 1：（2010-03-13）以下 C 语言用户标识符中，不合法的是（D）

A)_1　　B)AaBc　　C)a_b　　D)a-b

4.知识点：数据类型

例 1：（2010-03-16）以下关于 C 语言数据类型使用的叙述中错误的是（D）

A)若要准确无误差的表示自然数，应使用整数类型

B)若要保存带有多位小数的数据，应使用双精度类型

C)若要处理如“人员信息”等含有不同类型的相关数据，应自定义结构体类型

D)若只处理“真”和“假”两种逻辑值，应使用逻辑类型

【解析】结构数据类型是根据已定义的一个或多个数据类型用构造的方法来定义的，一个结构类型的值可以分解成若干个“成员”或“元素”，每个“成员”都是一个基本数据类型或又是一个构造类型，C 项正确。C 语言没有逻辑类型，故 D 项错误。

例 2：（2011-03-26）定义无符号整数类为 UInt,下面可以作为类 UInt 实例化值的是（B）

A)-369　　B)369　　C)0.369　　D)整数集合{1,2,3,4,5}

【解析】无符号整数类型为只有正整数和 0，所有选择 B。

5.知识点：常量

- 整型常量

十进制整常数：没有前缀，其数码为 0-9。例如 65535，-568

八进制整常数：以 0 开头，其数码为 0-7。例如 015

十六进制整常数：以 0X 或 0x 开头，其数码为 0-9，A-F 或 a-f。例如 0x2A

长整型数：整型常数后缀为 L 或 l。例如 358000L

无符号数：整型常数后缀为 U 或 u。例如 158u

- 实型常量的形式

小数形式：必须有小数点，小数点前后不能同时没有数字。

指数形式：aEn（其中 a 表示十进制数，E 阶码标志，n 阶码）。**E 前 E 后必须有数字，E 后必为整数，可以带符号。**例如 2.1E-3、5E8

例 1：（2011-09-13）以下选项中关于 C 语言常量的叙述错误的是：**（D）**

A.所谓常量，是指在程序运行过程中，其值不能被改变的量

B.常量分为整型常量、实型常量、字符常量和字符串常量

C.常量可分为数值型常量和非数值型常量

D.经常被使用的变量可定义成常量

【解析】根据常量的基本概念，则不难判断 A、B、C 均正确。

例 1：（2009-04-13）以下选项中，能用作数据常量的是（D）

A)o115　　B)0118　　C)1.5e1.5　　D)115L

【解析】八进制整常数以 0 开头，A 项错误；八进制整常数的数码味 0-7，B 项错误；实型常量的指数形式 E 后必为整数，故 C 项错误；长整型数后缀为 L 或 l，故 D 项正确。

例 2：（2010-03-26）以下选项中可用作 C 程序合法实数的是（ A）

A).1e0　　B)3.0e0.2　　C)E9　　D)9.12E

【解析】实型指数表示方式“E 前 E 后比有数，E 后必须是整数”，故选 A

- 字符常量：用**单引号‘’括起来**的一个字符。包括普通字符常量和转义字符。

普通字符常量：如‘a’、‘A’、‘1’。

转义字符：由一对单引号引起来的以\开头的若干字符的组合。如‘\n’表示回车换行、‘\\’表示反斜线符、‘\’’表示单引号符、‘\”’表示双引号、‘\ddd’表示八进制数、‘\xhh’表示十六进制数。

例 3：（2010-09-15）以下选项中能表示合法常量的是（C）

A)整数：1,200　　B)实数：1.5E2.0　　C）字符斜杠：‘\\’　　D）字符串：

“\007”

【解析】表示整常量不能用逗号分隔，A 项错误；实型常量的指数形式 E 后必为整数，故 B 项错误；’\007’是转义字符，只能用单引号括起来，不能用双引号或其他括号，故 D 项错误。例 4：（2010-03-26）以下选项中不能用作 C 程序合法常量的是（ B）

A)1,234　　B)'\123'　　C)123　　D)"\x7G"

【解析】A 中的‘,’在编译时会出错的，直接写成 1234 即可，B 是一个三位八进制数代表的字符，C 是一个数字常量，D 是一个字符串，其中\x7 是一个一位十六进制数代表的字符，也即 D 中的字符串是两个字符，如果 sizeof 的话是 3，如果 strlen 的话是 2。

6.知识点：变量

- 整型变量和实型变量

以考试标准 Viusal c++6.0 为例，说明各类型变量所占的位数。

类型	所占位数	所占字节数		类型	所占位数	所占字节数
short	16	2		unsigned int	32	4
int	32	**4**		unsigned long	32	4
long	32	4		float	32	4
unsigned short	16	2		double	64	**8**

例 1：（2010-09-38）有以下程序

```
#include <stdio.h>
main(0)
{int s,t,A=100;double B=6;
s=sizeof(A); t=sizeof(B);
printf(“%d,%d\n”,s,t);
}
```

在 VC6 平台上编译运行，程序运行后的输出结果是(C)

A)2,4　　B)4,4　　C)4,8　　D)10,6

【解析】在 VC6 平台上，int 型占 4 个字节，double 型占 8 个字节，故答案是 C。

- 字符变量：一个字符变量只能存放一个字符。字符串不能存放在字符变量中，只能用字符数组或指针存放字符串。

例 1：（2009-09-22）有以下定义语句，编译时会出现编译错误的是(C)

A)char a=’a’；　　B)char a= ‘\n’；　　C）char a= ‘aa’；　　D）char a= ‘\x2d’；

【解析】一个字符变量只能存放一个字符，A 项字符变量存放的是普通字符常量；B、D 项字符变量存放的转义字符；字符常量只能是单个字符，不能是字符串，’aa’不是单个字符，故 C 项错误。

7.知识点：ASCII 码

- ‘0’的 ASCII 码值为 48；‘A’的 ASCII 码值为 65；‘a’的 ASCII 码值为 97。
- 大写字母与小写字母的 ASCII 码值相差 32。

例 1：（2009-09-23）有以下程序

```
#include <stdio.h>
main()
{char c1,c2;
c1=’A’+’8’-‘4’;
c2=’A’+’8’-‘5’;
printf(“%c,%d\n”,c1,c2);
}
```

已知字母 A 的 ASCII 码为 65，程序运行后的输出结果是（A）

A)E,68　　B)D,69　　C)E,D　　D)输出无定值

【解析】'A'的 ASCII 码值为 65，'8'的 ASCII 码值为 56，'4'的 ASCII 码值为 52,65+56-52=69，对应于字符'E'。'5'的 ASCII 码值为 53,65+56-53=68，对应于字符'D'。字符变量 c1 以%c 格式输出，即'E'。字符变量 c2 以%d 格式输出，即 68。故答案是 A。

8.知识点：进制转换

- 十进制转换成二进制、八进制、十六进制：将十进制的数字除以 2（8、16），得到的商数继续除以 2，直到商为 0，然后将各次相除所得的余数从后往前排列。
- 二进制、八进制、十六进制转换成十进制：将二（八、十六）进制数的每一位数从高位到低位乘以 2 的 n-1 次幂，n 为该位所在的位数。
- 二进制与八进制之间的转换：每个八进制数字转换成 3 位二进制数字。
- 二进制与十六进制之间的转换：每个十六进制数字转换成 4 位二进制数字。

例 1：（2010-09）以下程序运行后的输出结果是____2008____。

```
#include <stdio.h>
main()
{int a=200,b=010;
printf（"%d%d\n",a,b）;
}
```

【解析】八进制数 010 转换成十进制数为 8，按"%d%d"格式分别输出 a、b，即 2008。

9.知识点：运算符的优先级

级别由大到小:(逻辑)非 >算(术运算符) >关(系运算符) >(逻辑)与 >(逻辑)或 >条(件运算符) >赋(值运算符)

10.知识点：强制类型转换运算符

- 一般形式：（类型说明符）（表达式）
- 功能：把表达式的运算结果强制转换成类型说明符所表示的类型

例 1：（2009-04-06）表达式(int)((double)(5/2)+2.5)的值是 4。

【解析】(int)((double)(5/2)+2.5)→(int)((double)2)+2.5）→(int)(2.000000+2.5)→(int)(4.500000) →4。

例 2：以下程序运行后的输出结果是 （） 。

```
main()
{    int a;
     a=(int)((double)(3/2)+0.5+(int)1.99*2);
     printf("%d\n",a);
}
```

答案：3

【 解 析 】 (3/2)=1 ， (double)(3/2)+0.5=1.5 ， (int)1.99*2=2 ， 所 以 (double)(3/2)+0.5+(int)1.99*2=3.5，故 a=3。

11.知识点：算术运算符： +、—、*、/、%

%：参与运算的量均为**整型**。

/：当除号左右两边都是整数的时候，商也要是**整数**，如果不是整数则舍弃小数部分。当参与运算量有一个为实型，则商为 double 型。

例 1：（2011-09-14）若有定义语句：int a=10;double b=3.14;,则表达式'A'+a+b 值得类型是 （C）

A.char　　B.int　　C.double　　D.float

【解析】各种类型数据混合运算时，最终结果的类型可依据以下转换规律：

char -> short -> int -> long -> float -> double

例 1：（2011-09-15）如有定义语句：int x=12,y=8,z;,在其后执行语句 z=0.9+x/y,则 z 的值为 （B）

A.1.9　　B.1　　C.2　　D.2.4

【解析】x,y,z 均为整型变量，因此 x/y 结果为 1，即 z=0.9+1=1.9，z 最终结果为 1

例 1：（2010-09-14）有以下定义：int a；long b；double x，y；则以下选项中正确的表达式是（A）

A)a%(int)(x-y)　　B)a=x!=y；　　C) (a*y)%b　　D)y=x+y=x

【解析】参与%运算的左右操作数均为整型，故 A 项正确，C 项错误。B 项是赋值语句，不是表达式。赋值运算符左边必须是变量，而不是表达式，而 D 项中 x+y=x，错误。

12.知识点：逗号表达式

- 一般形式：表达式 1，表达式 2……表达式 n
- 求值过程：自左向右依次求解，最后一个表达式的值为**整个**逗号表达式的值。

例 1：（2009-04-14）设有定义：int x=2；以下表达式中，值不为 6 的是(D)

A)x*=x+1　　B)x++, 2*x　　C)x*=(1+x)　　D)2*x, x+=2

【解析】A 项 x=x*(x+1)=2*(2+1)=6；B 项 x++后，x 的值为 3,2*x 的值为 6，整个逗号表达式的值为 6；C 项 x=x*(1+x)=2*3=6；D 项 x=x+2=2+2=4，整个逗号表达式的值为 4。故答案是 D。

13.知识点：自增自减运算符

++i　i 自加 1 后再参与运算

i++　i 参与运算后再自加 1

--i　i 自减 1 后再参与运算

i--　i 参与运算后再自减 1

++和– –的运算对象只能为变量，不能是常量或表达式。

例 1：（2009-09-06）若有定义语句：int a=5；则表达式：a++的值是 5 。

【解析】表达式 a++的值是 a 未加 1 之前的值，即 5。.

- 当++或—与 printf 语句结合时，若++/--x 的形式，则先自增/自减，然后输出；若 x++/--的形式，则先输出 x 原值，在进行自增/自减操作。

例 2：（2011-03-26）有以下程序

```
#include
main()
{    int x=011;
     printf("%d\n",++x);
}
```

程序运行后的输出结果是（C）

A)12　　B)11　　C)10　　D)9

【解析】011 是一个八进制的数，转为十进制则为 1*8+1=9；++x 是 x 先自增 1 再参与其他运算，所以先 x 自增 1 为 10 再输出即为 10，故选 C。

14.知识点：赋值表达式

- 一般形式：变量名=表达式
- **赋值运算符左边必须是变量，而不是表达式。**

例 1：（2010-03-14）若有定义：double a=22;int i=0,k=18；则不符合 C 语言规定的赋值语句是（ C）

A)a=a++，i++；　　B)i=（a+k）<=(i+k)；　　C)i=a%11；　　D)i=!a；

【解析】a 为 double 型，参与%运算的操作数均为整型，故 C 项错误。

例 2：（2011-03-26）若有定义语句：int a=3,b=2,c=1;，以下选项中错误的赋值表达式是（A）

A)a=(b=4)=3;　　B)a=b=c+1;　　C)a=(b=4)+c;　　D)a=1+(b=c=4);

【解析】赋值表达式的左边必须是变量不能是表达式。而答案 A 中，（b =4）是一个赋值表达式。

- 复合赋值运算符和表达式

+=、—=、*=、/=、%=、<<=、>>=、 &=、 ^=、 |=

例 2：（2010-09-16）表达式 a+=a-=a=9 的值是（D）

A)9 B)-9 C)18 D)0

【解析】赋值运算符的结合性是从右到左，a=9→a=a-9=0→a=a+0=0+0=0。故答案是 D。

15.知识点：位运算

- 按位与运算&

规则：0&0=0 0&1=0 1&0=0 1&1=1

- 按位或运算|

规则：0|0=0 0|1=1 1|0=1 1|1=1

- 按位异或运算∧

规则：0∧0=0 0∧1=1 1∧0=1 1∧1=0

- 求反运算~

规则：~0=1 ~1=0

- 左移运算<<

规则：把<<左边的运算数的各二进位全部左移若干位，高位丢弃，低位补 0。**左移一位相当于该数乘以 2。**左移 n 位相当于该数乘以 2^n。

例 1：（2010-09-40）有以下程序

```
#include <stdio.h>
main()
{short c=124;
c=c____D____;
printf("%d\n",c);
}
```

若要使程序的运行结果为 248，应在下划线处填入的是

A)>>2 B)|248 C)&0248 D)<<1

【解析】左移一位相当于该数乘以 2，124>>1 相当于 124 乘以 2，等于 248。故答案是 D。

例 2：（2009-04-39）有以下程序

```
#include <stdio.h>
main()
{int a=5,b=1,t;
t=(a<<2)|b;
printf("%d\n",t);
}
```

程序运行后的输出结果是（A）

A)21 B)11 C)6 D)1

【解析】左移 2 位相当于该数乘以 2^2。5<<2 相当于 5 乘以 4 等于 20。20|1=21。故答案是 A。

- 右移运算>>

把>>左边的运算数的各二进位全部右移若干位。高位补 0，低位丢弃。**右移一位相当于该数除以 2。**右移 n 位相当于除以 2^n。

例 3：（2009-09-39）若有以下程序段

```
int r=8;
printf("%d\n",r>>1);
```

输出结果是（C）

A)16　　B)8　　C)4　　D)2

【解析】右移一位相当于该数除以 2。8>>1 相当于除以 2，等于 4，故答案是 C。

16.知识点：关系运算符和表达式

例 1：（2010-09-08）在 C 语言中，当表达式值为 0 时表示逻辑值“假”，当表达式值为_____

非 0_____时表示逻辑值“真”。

例 2：（2010-03-06）设 x 为 int 型变量，请写出一个关系表达式(x%3==0)&&(x%7==0)_____，用以判断 x 同时为 3 和 7 的倍数时，关系表达式的值为真。

17.知识点：逻辑运算符和表达式

- 逻辑运算符优先级从高到低：!、&&、||
- 逻辑运算的真值表

a	b	a&&b	a||b	!a
真	真	真	真	假
真	假	假	真	假
假	真	假	真	真
假	假	假	假	真

- 进行逻辑运算时，**若&&左边的值为 0，则不再对右边的运算对象进行运算，整个表达式的值为 0。**

例 1：（2011-09-17）**若有定义语句：int k1=10,k2=20;,执行表达式（k1=k1>k2）&&(k2=k2>k1)后，k1 和 k2 的值分别为 （B）**

A.　0 和 1　　B.0 和 20　　C.10 和 1D.　10 和 20

【解析】k1>k2 为假，因此 k1= k1>k2 结果 0，逻辑与左边表达式为假，右边表达式不再处理，因此 k1 结果为 0，k2 不变，仍为 20

例 2：int a=2；则表达式(!a==1)&&(a++==2)的值为 0，a 的值为 2。

【解析】! a 为 0，! a==1 的值为 0，&&左边的值为 0，则不再对右边的运算对象进行运算，所以 a 的值为 2。

- **若||左边的运算对象的值为 1，则不再对右边的运算对象进行运算，整个表达式的值为 1。**

例 2：（2010-03-17）若 a 是数值类型，则逻辑表达式(a==1)||(a!=1)的值是（A）

A)1　　B)0　　C)2　　D)不知道 a 的值，不能确定

【解析】a 的值有两种情况：a 等于 1 或 a 不等于 1。故逻辑表达式(a==1)||(a!=1)的值为 1。

18.知识点：格式输入函数 printf()

(1)一般形式：printf(“格式控制字符串”,输出列表)；

(2)遇到控制字符，按照控制字符的规定输出，**遇到非格式字符串按原样输出**。

例 1：（2009-04-07）若变量 x,y 已定义为 int 类型且 x 的值为 99，y 的值为 9，请将输出语句 printf(“ x/y=%d ”,x/y)；补充完整，使其输出的计算结果形式为：x/y=11。

例 2：（2009-04-15）程序段：int x=12;double y=3.141593;printf(“%d%8.6f”,x,y)；的输出结果是（A）

A)123.141593　　B)12 3.141593　　C)12,3.141593　　D)123.1415930

【解析】以%8.6f 格式输出，即输出的总宽度为 8，小数的位数为 6。

19.知识点：格式输入函数 scanf()

（1）一般形式：scanf(“格式控制字符串”,地址列表)；

（2）若格式控制字符串中有非格式字符串，则按原样输入。若格式控制字符串中

没有非格式字符作输入数据之间的间隔，则可用空格、Tab、回车作间隔。

例 1：(2011-09-16)若有定义：int a,b;,通过语句 scanf(“%d;%d”,&a,&b);,能把整数 3 赋给变量 a,5 付给变量 b 的输入数据是

A.3　5　B.3,5　　C.3；5　D.35

【解析】scanf 格式字符串中两个%d 之间带有非格式字符分号，运行时应原样输入。

例 1：(2005-04-19)有以下程序

```
main()
{int m,n,p;
scanf(“m=%dn=%dp=%d”,&m,&n,&p);
printf(“%d%d%d\n”,m,n,p);
}
```

若想从键盘上输入数据，使变量 m 中的值为 123，n 中的值为 456，p 中的值为 789，则正确的输入是（A）

A)m=123n=456p=789　　B)m=123　n=456　p=789

C)m=123,n=456,p=789　　D)123　456　789

【解析】scanf(“m=%dn=%dp=%d”,&m,&n,&p); 格式控制字符串中有非格式字符串，则按原样输入，故答案是 A。

例 2：(2011-03-26)有以下程序段

```
char name[20];
int num;
scanf("name=%s num=%d",name;&num);
```

当执行上述程序，并从键盘输入：name=Lili num=1001<回车>后，name 的值（A）

A)Lili　　B)name=Lili　　C)Lili num=　　D)name=Lili num=1001

【解析】scanf 中格式字符串的非格式字符串原样输入，如其中的‘name=’和‘num=’，所以接受到的 name 值是后面的 Lili，num 值为 1001。故选 A。

（3）在输入字符数据时，若格式控制字符串中无非格式字符，则认为所有输入的字符均为有效字符。**空格会被当作字符赋值给变量。**

例 3：（2009-04-23）有以下程序

```
#include <stdio.h>
main()
{int a1,a2;　char c1,c2;
scanf(“%d%c%d%c”,&a1,&c1,&a2,&c2);
printf(“%d,%c,%d,%c”,a1,c1,a2,c2);
}
```

若想通过键盘输入，使得 a1 的值为 12，a2 的值为 34，c1 的值为字符 a，c2 的值为字符 b，程序输出结果是：12，a，34，b 则正确的输入格式是（以下_代表空格，<CR>代表回车）

A)12a34b<CR>　B)12_a_34_b<CR>　C)12,a,34,b<CR>　　D)12_a34_b<CR>

【解析】空格会被当作字符赋值给变量，故 B、D 错误；C 项中逗号会被当作字符赋值给变量，故 C 项错误。

（4）格式字符串

- 一般形式：%[*][输入数据宽度][长度]类型
- *表示该输入项，读入后不赋予相应的变量。
- 输入数据宽度：用十进制数指定输入的字符数。

例 4：（2010-09-07）有以下程序

```
#include <stdio.h>
main()
{int x,y;
scanf(“%2d%ld”,&x,&y); printf(“%d\n”x+y);
}
```

程序运行时输入：1234567 程序的运行结果是____34579_____。

【解析】%2d 输入的数据宽度为 2，x 的值为 12，y 的值为 34567，x+y=34579。

例 5：（2009-04-16）若有定义语句：double x,y,*px,*py；执行了 px=&x；py=&y；之后，正确的输入语句是(C)

A) scanf(“%f%f”,x,y); B)scanf(“%f%f”,&x,&y);

C)scanf(“%lf%le”,px,py); D)scanf(“%lf%lf”,x,y);

【解析】A、D 项地址列表，应为&x,&y；x、y 为 double 型，格式控制字符串不是%f%f，故 B 项错误。

- 当定义的变量类型和 scanf 中“格式符”类型不一致时（这里只限定在字符型和整型两种类型，其他数据类型不可以。）整型和字符型之间通过 ASCALL 可以相互转化。

例 6：（2011-03-12）有以下程序 (说明：字符 0 的 ASCII 码值为 48)

```
main()
{   char c1,c2;
    scanf("%d",&c1);
    c2=c1+9;
    printf("%c%c\n",c1,c2);
}
```

若程序运行时从键盘输入 48<回车>，则输出结果为 （09）。

【解析】输入 48，则 c1=48，c2=48+9，以字符输出 c1 和 c2，则为输出 ASCII 码为 48 和 57 对应的字符，分别为 0 和 9。

20.知识点：字符输出函数 putchar()

一般形式：putchar(‘a’)；putchar(‘\101’)；

21.知识点：字符输入函数 getchar()

一般形式：getchar()；或 ch=getchar()；

例 1：（2010-03-15）有以下程序

```
#include <stdio.h>
main()
{char a,b,c,d;
scanf(“%c%c”,&a,&b);
c=getchar();  d=getchar();
printf(“%c%c%c%c\n”,a,b,c,d);
}
```

当执行程序时，按下列方式输入数据（从第 1 列开始，<CR>代表回车，注意：回车也是一个字符）

12<CR>

34<CR>

则输出结果是(B)

A)1234 B)12 C)12 D)12

3 34

【解析】以%c 格式读取一个字符，以 getchar()形式也是读取一个字符。空格会被当作字符赋值给变量。所以，a=’1’,b=’2‘,c=’\n’,d=’3‘。

- getchar 经常作为 while 循环条件，判断输入哪一个字符时，循环结束。例如，输入字符串，直到输入‘0’，程序结束，则可写成 **while(getchar()!='0')**

例 2：（2011-03-26）有以下程序段

```
main()
{   …
    while( getchar()!='\n');
    …
}
```

以下叙述中正确的是（C）

A)此 while 语句将无限循环

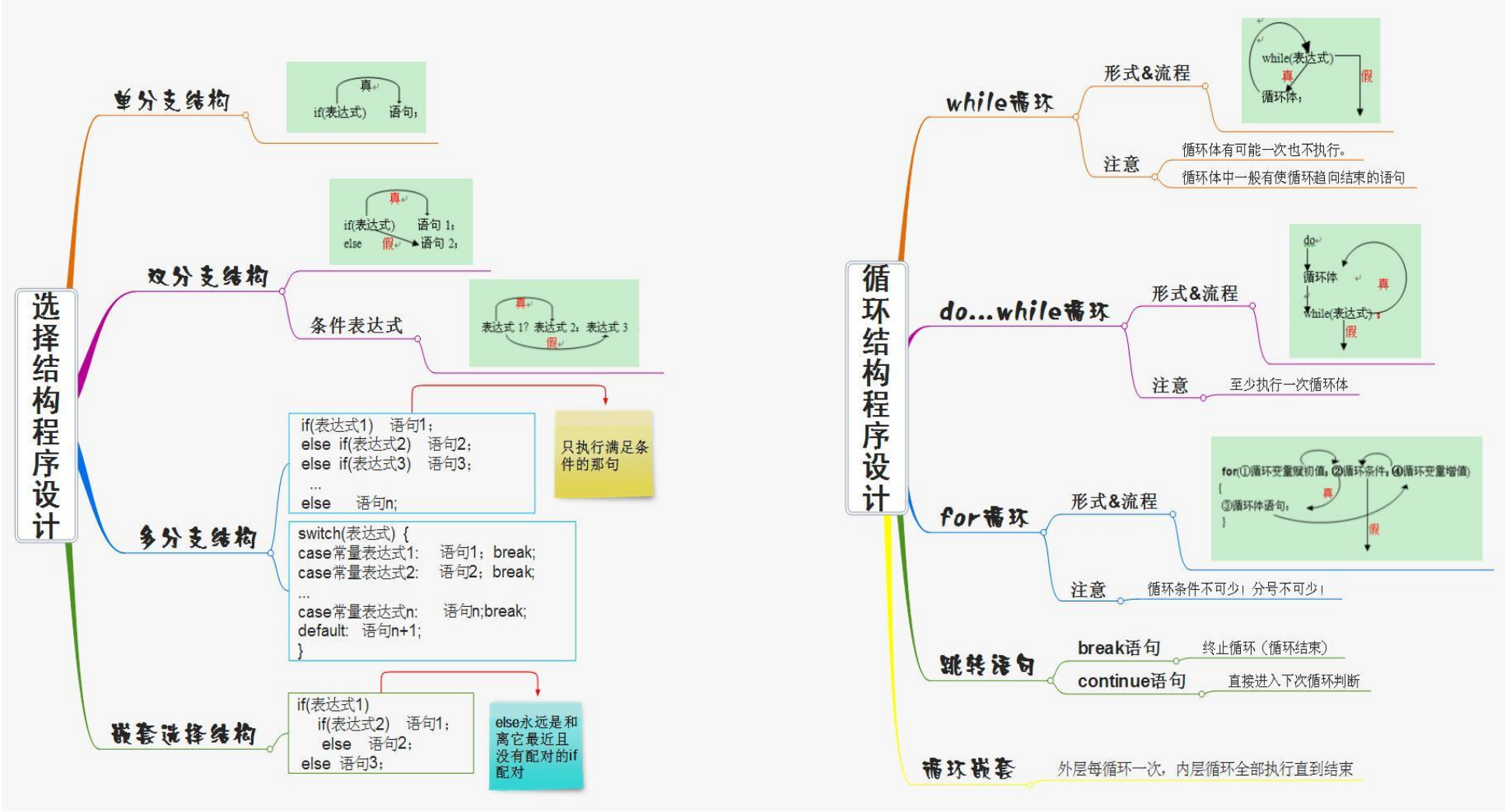
B) getchar()不可以出现在 while 语句的条件表达式中

C)当执行此 while 语句时，只有按回车键程序才能继续执行

D)当执行此 while 语句时，按任意键程序就能继续执行

【解析】while 条件表达式 getchar()!='\n'只要输入的字符不是回车，就一直执行循环，而循环内是一个空语句，所以按回车键程序才能继续执行，选择 C。

第二课 选择结构与循环结构



本课重要知识点：

1.知识点：if 语句

- if（表达式），其中表达式可以是任意合法的 C 语言表达式。

例 1：（2011-09-19）下列条件语句中输出结果与其他语句不同的是（D）

A.if（a） printf(“%d\n”,x); else printf(“%d\n”,y);

B.if（a==0） printf(“%d\n”,y); else printf(“%d\n”,x);

C.if（a!=0） printf(“%d\n”,x); else printf(“%d\n”,y);

D.if（a==0） printf(“%d\n”,x); else printf(“%d\n”,y);

【解析】作为 if 语句的条件，a 等价于 a!=0，因此 A 和 C 是等价的。

例 2：（2011-03-26）if 语句的基本形式是：if(表达式)语句，以下关于“表达式”值的叙述中正确的是（D）

A)必须是逻辑值 B)必须是整数值 C)必须是正数 D)可以是任意合法的数值

【解析】表达式不限于关系表达式、逻辑表达式，也可以是任意的数值类型，其中非 0 为真，0 为假。故选 D。

- 对于单个 if 语句，其后跟随的受 if 控制的只能是 1 个语句或者是 1 个复合语句。

例 1：（2009-09-17）设有定义：int a=1,b=2,c=3;以下语句中执行效果与其它三个不同的是（C）

A)if(a>b) c=a，a=b，b=c； B)if(a>b){ c=a，a=b，b=c；}

C)if(a>b) c=a；a=b；b=c； D)if(a>b){ c=a；a=b；b=c；}

【解析】对于单个 if 语句，其后跟随的受 if 控制的只能是 1 个语句或者是 1 个复合语句。故 C 项，if(a>b)条件成立时，语句 c=a;才被执行，而无论 if(a>b)条件成立与否，语句 a=b;b=c;都会被执行。A、B、D 项，只有 if(a>b)条件成立时，语句 c=a;a=b;b=c; 才被执行。

例 2：（2011-03-26）以下程序运行后的输出结果是 ____200____

```
main()
{   int x=10,y=20,t=0;
    if(x==y)t=x;x=y;y=t;
    printf("%d %d\n",x,y);
}
```

【解析】x=10,y=20，所以 x==y 为 false，t=x;不执行，直接执行 x=y;y=t;x 为 20，y 为 0。注意和 if(x==y){t=x;x=y;y=t;}区别。

- **当多个 if(表达式)单分支形式并行出现时**，其执行顺序是：先判断第一 if 单分支的条件，如果为真，执行其后面的语句，执行完后，继续向下执行。判断第二个 if 单分支语句，依次类推。如果第一个 if 表达式的值为假，则不执行其后的语句，而去判断第二个 if 单分支条件。

例 1：（2011-03-26）有以下程序

```
main()
{   int x;
    scanf("%d",&x);
    if(x>15) printf("%d",x-5);
    if(x>10) printf("%d",x);
    if(x>5) printf("%d\n",x+5);
}
```

若程序运行时从键盘输入 12<回车>，则输出结果为 ____1217____

【解析】输入 12，比较不大于 15 所以不执行 if(x>15)内的语句；大于 10 执行 printf("%d",x);输出 12；大于 5 执行 printf("%d",x+5);输出 17。

2.知识点：if 语句的嵌套

- 内嵌结构中，**else 总是与前面最近的且未曾配对的 if 语句配对，组成一对 if-else 语句。**

例 2：（2010-09-18）有以下程序

```
#include <stdio.h>
main()
{int a=1,b=0;
```

if(!a) b++;

else if(a==0) if(a) b+=2;

else b+=3;

printf("%d\n",b);

}程序运行后的输出结果是（A）

A)0 B)1 C) 2 D)3

【解析】else 总是与前面最近的且未曾配对的 if 语句配对，组成一对 if-else 语句。即 else b+=3;

与 if(a) b+=2;配对。if(!a)不成立，不执行 b++;else if(a==0)不成立，不执行 if(a) b+=2; else b+=3;故直接执行 printf("%d\n",b);，输出结果为 0。

3.知识点：条件表达式

- 一般形式：表达式 1？表达式 2：表达式 3
- 执行过程：表达式 1 的值为真，条件表达式取表达式 2 的值；表达式 1 的值为假，条件表达式取表达式 3 的值。

例 1：（2009-09-19）以下程序段中，与语句：k=a>b?(b>c?1:0):0；功能相同的是（A）

A)if((a>b)&&(b>c)) k=1;
  else k=0;

B) if((a>b)||(b>c)) k=1;
  else k=0;

C) if(a<=b) k=0;
  else if(b<=c) k=1;

D) if(a>b) k=1;
  else if(b>c) k=1;
  else k=0;

【解析】条件运算符的结合性是从右至左。当 b>c 时，b>c?1:0 的值为 1，k=a>b?1:0，同时当 a>b 时，a>b?1:0 的值为 1，此时 k=1。否则 k=0。故答案为 A。

4.知识点：switch 语句

- switch 语句的圆括号中的**表达式的值必须是整型或字符型**，不能为实型。
- case 后的表达式可以是求得**整型和字符型的常量表达式**，但不能含有变量。
- default 的位置对程序的执行没有影响。

例 1：（2010-09-19）若有定义语句 int a,b;double x;则下列选项中没有错误是（C）

A)switch(x%2)
{case 0:a++;break;
 case 1:b++;break;
 default:a++;b++;
}

B) switch((int)x%2.0)
{case 0:a++;break;
 case 1:b++;break;
 default:a++;b++;
 }

C)switch((int)x%2)
{case 0:a++;break;
 case 1:b++;break;
 default:a++;b++;
}

D)switch((int)(x)%2)
{case 0.0:a++;break;
 case 1.0:b++;break;
 default:a++;b++;
 }

【解析】switch 语句的圆括号中的表达式的值必须是整型或字符型，不能为实型。x 是 double 型，x%2 不合法，A 项错误。(int)x%2.0 不合法，B 项错误。(int)x%2 值为整型，故 C 项正确。case 后面的表达式可以是求得整型量和字符型量的常量表达式，故 D 项错误。

例 2：（2009-09-18）有以下程序

#include <stdio.h>

main()

{int c=0,k;

for(k=1;k<3;k++)

```
switch(k)
{default:c+=k;
case 2:c++;break;
case 4:c+=2;break;
}
printf(“%d\n”,c);
```

}程序运行后的输出结果是(A)

A)3　　B)5　　C)7　　D)9

【解析】当 k=1 时，没有匹配的 case，执行 default 语句，c=c+k=0+1=1。继续往下执行 case 2 后边的语句，c++，c 变为 2，遇到 break 语句，于是退出内层 switch 结构。当 k=2 时，找到匹配的 case 2，执行 c++; c 变为 3，遇到 break 语句，于是退出内层 switch 结构。执行外层循环 for 结构的 k++，k 变为 3，k<3 不成立，跳出 for 语句。最后输出结果为 3。

- 当 switch 语句和循环语句（for、while）时，要注意 switch 里面的 break 语句退出的位置。解决这一问题的方法是：首先划分结构，确定循环的语句体包括哪些语句。然后，在继续划分循环体内的语句，如果循环体内的语句只有一条 switch 语句，其内部出现了 break,**遇到 break，要结束当前 switch 语句的执行，继续执行下一次循环**。

例 1：（2011-03-26）有以下程序

```
main()
{    int s;
     scanf("%d",&s);
     while(s>0)
     { switch(s)
          { case1:printf("%d",s+5);
            case2:printf("%d",s+4); break;
            case3:printf("%d",s+3);
            default:printf("%d",s+1);break;
            }
            scanf("%d",&s);
     }
}
```

运行时，若输入 1 2 3 4 5 0<回车>，则输出结果是（A）

A)6566456　　B)66656　　C)66666　　D)6666656

【解析】while 循环 s<=0 时退出循环，输入 1 2 3 4 5 0，只有当输入 0 时退出循环，switch 中当 s=1 时，执行 case 1，case 2，由于 case 1 后面没有 break，会继续执行 case 2，遇到 break，退出 switch；当 s=2 时，执行 case 2，退出 switch；当 s=3 时，执行 case 3，default，由于 case 3 后面没有 break，会继续执行 default，遇到 break，退出 switch；当 s=4 和 5 时，执行 default，退出 switch。所以输入 1 时，输出 65；输入 2 时，输出 6；输入 3 时，输出 64；输入 4 时，输出 5；输入 5 时，输出 6。故选择 A。

5.知识点：while,do-while,for 三种循环结构

例 1：（2011-09-20）有以下程序：

```
#include <stdio.h>
main()
{  int a=7;
 while(a--);
printf(“%d\n”,a);
```

}程序运行后的输出结果是 (B)

A.－1　　B.0　　C.1　　D.7

【解析】此类题一定要注意标点符号的位置，while(a--);该行末尾的分号即为一条空语句，来作为 while 的循环体。

例 1：（2010-09-20）有以下程序

```
#include <stdio.h>
main()
```

{int a=1,b=2;

while(a<6) {b+=a;a+=2;b%=10;}

printf(“%d,%d\n”,a,b);

}程序运行后的输出结果是(B)

A)5,11　　B)7,1　　C)7,11　　D)6,1

例 2：（2011-03）有以下程序段

```
main()
{   …
    while( getchar()!='\n');
    …
}
```

以下叙述中正确的是(C)

A)此 while 语句将无限循环

B) getchar()不可以出现在 while 语句的条件表达式中

C)当执行此 while 语句时，只有按回车键程序才能继续执行

D)当执行此 while 语句时，按任意键程序就能继续执行

【解析】while 条件表达式 getchar()!='\n'只要输入的字符不是回车，就一直执行循环，而循环内是一个空语句，所以按回车键程序才能继续执行，选择 C。

- **循环体部分可以只有一条空语句，不做任何操作。**

例 3：（2010-09-21）有以下程序

#include <stdio.h>

main()

{int y=10;

while(y--);

printf(“y=%d\n”,y);

}程序执行后的输出结果是(B)

A)y=0　　B)y=-1　　C)y=1　　D)while 构成无限循环

【解析】while(y--)后面的分号是一个空语句，当 y--不等于 0 时执行空语句，当 y—等于 0 时，执行 printf 函数输出 y 值，当 y—等于 0 时退出循环，此时 y 值变成-1。

- 如果第一次判断表达式的值为 0，则循环一次也不执行。即 **while 循环体最少执行 0 次。**

例 4：（2011-03）当执行下列程序时，输入 1234567890<CR>，则其中 while 循环体将执行_____0_____次。

#include <stdio.h>

main()

{char ch;

while((ch=getchar()==’0’))　　printf(‘#’);

}

【解析】ch=getchar()，ch 第一次读入的值为’1’，故 while((ch=getchar()==’0’))不成立，循环一次也不执行。

- **while 循环与 switch 结构**

例 5、（2011-03）有以下程序

```
main()
{   int s;
    scanf("%d",&s);
    while(s>0)
    { switch(s)
        { case1:printf("%d",s+5);
        case2:printf("%d",s+4); break;
        case3:printf("%d",s+3);
        default:printf("%d",s+1);break;
        }
```

```
        scanf("%d",&s);
    }
}
```

运行时，若输入 1 2 3 4 5 0<回车>，则输出结果是(A)

A)6566456　　B)66656　　C)66666　　D)6666656

【解析】: while 循环 s<=0 时退出循环，输入 1 2 3 4 5 0，只有当输入 0 时退出循环，switch 中当 s=1 时，执行 case 1，case 2，由于 case 1 后面没有 break，会继续执行 case 2，遇到 break，退出 switch；当 s=2 时，执行 case 2，退出 switch；当 s=3 时，执行 case 3，default，由于 case 3 后面没有 break，会继续执行 default，遇到 break，退出 switch；当 s=4 和 5 时，执行 default，退出 switch。所以输入 1 时，输出 65；输入 2 时，输出 6；输入 3 时，输出 64；输入 4 时，输出 5；输入 5 时，输出 6。故选择 A。

6.知识点：do-while 语句

例 1：（2009-09-09）以下程序运行后的输出结果是_____5_____。

```
#include <stdio.h>
main()
{int a=1,b=7;
do{
b=b/2; a+=b;
}while(b>1);
printf(“%d\n”,a);}
```

- do-while 循环体最少执行 1 次。

例 2：（2007-09）若变量已正确定义，有以下程序段

```
i=0;
do printf(“%d,”,i); while(i++);
printf(“%d\n”,i);
```

其输出结果是 （B）

A)0,0　　B)0,1　　C)1,1　　D)程序进入无限循环

【解析】执行 do-while 循环，输出 i 的值为 0，接着判断循环条件 i++，i++的值为 0，i 的值为 1，故答案是 B。

7.知识点：for 语句

例 1：（2011-03）有以下程序

```
main()
{   char b,c; int i;
    b='a'; c='A';
    for(i=0;i<6;i++)
    {   if(i%2)  putchar(i+b);
        else  putchar(i+c);
    }
    printf("\n");
}
```

程序运行后的输出结果是（B）

A)ABCDEF　　B)AbCdEf　　C)aBcDeF　　D)abcdef

【解析】: for 循环执行 6 次，每次判断 i 能否被 2 整除，如果不能，则输出 i+b，如果可以则输出 i+c，注意 if(i%2)是求表达式 i%2 是否为 true，即其结果是不是非 0。所以 i 为 0 时，输出 0+A=A，i 为 1 时，输出 1+a=b，i 为 2 时，输出 2+A=C，i 为 3 时，输出 3+a=d，i 为 4 时，输出 4+A=E，i 为 5 时，输出 5+a=f。故选 B。

- 表达式 1、表达式 2、表达式 3 可以缺省，但两个分号不能省略。

表达式 1 省略的情况如下：

例 2：（2010-03-21）有以下程序

```
#include <stdio.h>
main()
{int a=1,b=2;
for(;a<8;a++) {b+=a;a+=2;}
printf(“%d,%d\n”,a,b);
```

}程序运行后的输出结果是(D)

A)9,18　　B)8,11　　C)7,11　　D)10,14

- **表达式 1 和表达式 3 也可以是逗号表达式。**

例 3：（2009-04-20）以下程序段中的变量已正确定义

```
for(i=0;i<4;i++,i++)
for(k=1;k<3;k++) printf("*");
```

程序段的输出结果是 (B)

A)********　　B)****　　C)**　　D)*

【解析】外层 for 循环，表达式 3 是逗号表达式，即 i 自加 2。.

- **for 循环与 switch 结构**

例 4：（2009-09-29）有以下程序

```
#include <stdio.h>
main()
{int a[]={2,3,5,4},i;
for(i=0;i<4;i++)
switch(i%2)
{case 0:switch(a[i]%2)
        {case 0:a[i]++;break;
         case 1:a[i]--;
        }break;
case 1:a[i]=0;
}
for(i=0;i<4;i++)  printf("%d",a[i]);  printf("\n");
```

}程序运行后的输出结果是(C)

A)3 3 4 4　　B) 2 0 5 0　　C)3 0 40　　C)0 3 0 4

例 5、（2011-03）有以下程序段

```
int i,n;
for(i=0;i<8;i++)
{   n=rand()%5;
    switch (n)
    {   case 1:
        case 3:printf("%d\n",n); break;
        case 2:
        case 4:printf("%d\n",n); continue;
        case 0:exit(0);
    }
    printf("%d\n",n);
}
```

以下关于程序段执行情况的叙述，正确的是（D）

A)for 循环语句固定执行 8 次

B)当产生的随机数 n 为 4 时结束循环操作

C)当产生的随机数 n 为 1 和 2 时不做任何操作

D)当产生的随机数 n 为 0 时结束程序运行

【解析】：rand()是随机产生一个整数，rand()%5 即随机得到 0-4 这五个数字，for 循环控制随机产生 8 次，当得到 1，3 时执行 printf("%d\n",n)并退出 switch，当得到 2，4 时执行 printf("%d\n",n)并继续循环，当得到 0 时，exit(0)正常结束程序。所以循环不一定执行 8 次，故选择 D。

- **for 循环与数组**

例 6：（2010-09-09）有以下程序

```
#include <stdio.h>
main()
{int i,n[]={0,0,0,0,0};
for(i=1;i<=4;i++)
```

{n[i]=n[i-1]*3+1； printf(“%d ”,n[i])；}

}程序运行后的输出结果是____1 4 13 40____。

- **注意区 赋值语句和判断语句**

例 7：（2011-03）若 i 和 k 都是 int 类型变量，有以下 for 语句

for(i=0,k=-1;k=1;k++) printf("*****\n");

下面关于语句执行情况的叙述中正确的是 （D）

A)循环体执行两次 B)循环体执行一次

C)循环体一次也不执行 D)构成无限循环

【解析】: for 循环的表达式 1 和表达式 3 可以是一个简单的表达式也可以是逗号表达式，本题表达式 1 是一个逗号表达式，i 赋值为 0，同时 k 赋值为-1。所以判断表达式 2：k=1 是 k 赋值为 1，故无限循环，选择 D。

8.知识点：循环嵌套

例 1：（2010-03-20）有以下程序

```
#include <stdio.h>
main()
{int i,j,m=1;
for(i=1;i<3;i++)
{  for(j=3;j>0;j--)
   {if(i*j>3)  break;
     m*=i*j;
    }
}
printf(“m=%d\n”,m);
```

}程序运行后的输出结果是 （A）

A)m=6 B)m=2 C)m=4 D)m=5

例 2：（2011-03）有以下程序

```
main()
{   int x=1,y=0;
    if(!x) y++;
    else if(x==0)
    if (x) y+=2;
    else y+=3;
    printf("%d\n",y);
```

}程序运行后的输出结果是 (D)

A)3 B)2 C)1 D) 0

【解析】: if-else 的嵌套，注意 else 总是与前面最近的且未配对的 if 语句配对组合。if(!x)中 x 为 1，！ x 为 0 为 false，即 if(!x) 后的语句不执行，else if(x==0)，x 为 1 也不等于 0，也不成立，故 else if 内的语句也不执行，而 if (x) y+=2;else y+=3;均为 else if 内的语句。所以直接输出 y 为 0，故选 D。

9.知识点：break 语句

- 功能：跳出整个 switch 语句；跳出循环体，防止死循环。
- break 语句只能出现在 switch、while、do-while、for 循环中，不能出现在其他地方。
- 作用范围：所在循环体从属的最内层循环，而不是外层的某个循环。

6.知识点：continue 语句

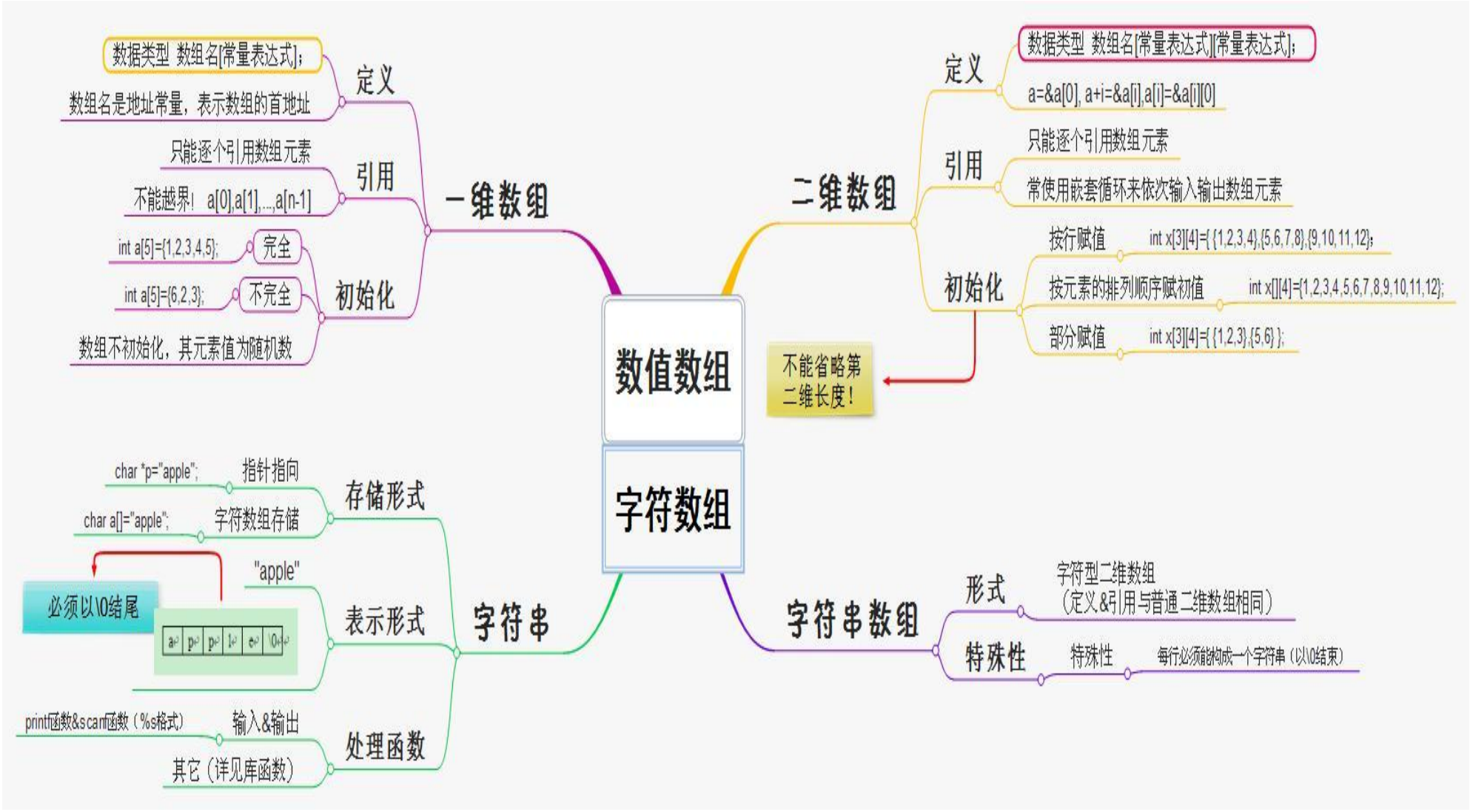
- 功能：结束本次循环，接着判断是否继续下一次循环。
- continue 语句只能出现在 while、do-while、for 循环中。
- 作用范围：所在循环体从属的最内层循环，而不是外层的某个循环。

例 1：（2009-04-09）以下程序运行后的输出结果是_____s=0_____

```
#include <stdio.h>
main()
{int k=1,s=0;
do{
      if((k%2)!=0)       continue;
      s+=k; k++;
   }while(k>10);
printf("s=%d\n",s);
}
```

【解析】k=1，if((k%2)!=0)条件成立，执行 continue 语句，结束本次循环，判断表达式 k>10，值为 0，故跳出 do-while 循环，执行 printf("s=%d\n",s);则输出结果为 s=0。

第三课 数组



本课重要知识点：

1.知识点：一维数组的定义

例 1：（2010-03-25）下列选项中，能正确定义数组的语句是（D）

A)int num[0…2008];　　B)int num[];

C)int N=2008;　　D)#define N 2008

int num[N];　　int num[N];

【解析】数组的长度必须是一个整型常量、整型常量表达式，故 A 项错误。定义数组时，不能省略数组长度，B 项错误。数组的长度不能是变量，C 项错误。数组的长度可以是符号常量，D 项正确。

例 2：(2010-09-28)若要定义一个具有 5 个元素的整型数组，以下错误的定义语句是（D）

A)int a[5]={0};　　B)int b[]={0,0,0,0,0};

C)int c[2+3];　　D)int i=5,d[i];

【解析】可以只给数组的前面一部分元素设定初值，A 项正确。对全部数组元素赋初值时，可以不指定数组元素的个数，B 项正确。数组的长度可以是一个整型常量表达式，C 项正确。数组的长度不能是变量，D 项错误。

2.知识点：一维数组元素的引用

例 1：(2008-09-21)若有定义语句：int m[]={5,4,3,2,1},i=4;则下面对 m 数组元素的引用中错误的是 （C）

A)m[--i]　　B)m[2*2]　　C)m[m[0]]　　D)m[m[i]]

【解析】m 数组元素的引用的下限为 0，上限为 4。i=4，--i 的值为 3，A 项正确。B 项没有越界，正确。m[0]=5，超过上限，已经越界，故 C 项错误。m[4]=1，没有越界，故 D 项正确。

例 2：(2011-03)以下程序运行后的输出结果是 __13715__

```
main()
{    int i,n[5]={0};
     for(i=1;i<=4;i++)
     { n[i]==n[i-1]*2+1; printf("%d",n[i]); }
     printf("\n");
}
```

【解析】：n[1]=1,n[2]=1*2+1=3,n[3]=3*2+1=7,n[4]=7*2+1=15

3.知识点：一维数组的初始化

- 顺序列出数组元素的全部初值，例如 int a[5]={1,2,3,4,5};
- 只给数组的前面一部分元素设定初值，未赋初值的元素自动取 0 值。例如，int a[5]={1,2};
- 对全部数组元素赋初值时，可以不指定数组元素的个数。例如，int a[]={1,2,3,4,5};

4.知识点：二维数组的初始化

- 按行分段给二维数组赋初值
- 按行连续赋值
- 对部分元素赋初值，未赋初值的元素自动取 0 值
- 若对全部元素赋初值，则第一维的长度可以不指定

例 1：（2006-04-16）以下数组定义中错误的是（B）

A) int x[][3]={0};

B) int x[2][3]={{1,2},{3,4},{5,6}};

C) int x[][3]={{1,2,3},{4,5,6}};

D) int x[2][3]={1,2,3,4,5,6};

【解析】若对全部元素赋初值，则第一维的长度可以不指定，C 项正确。二维数组的初始化可以按行连续赋值，D 项正确。C 项二维数组的初始化超过第一维度长度，故错误。

5.知识点：二维数组和 for 语句

例 1：（2010-03-09）有以下程序

```
#include <stdio.h>
main()
{int i,j,a[][3]={1,2,3,4,5,6,7,8,9};
for(i=0;i<3;i++)
for(j=i;j<3;j++)    printf("%d",a[i][j]);
printf("\n");
}
```

程序运行后的输出结果是_____123569_____。

6.知识点：字符串

定义：由“ ”引起来的，由若干个字符所组成的序列。

字符串的结束标志：'\0'

如字符串“china”，在内存中存储形式是

c	h	i	n	a	\0

该字符串的长度为 5 字节，所占的存储空间为 6 字节。

7.知识点：字符数组的定义及初始化

- 字符数组名是地址常量，不能赋值给数组名。

例 1：（2009-04-32）以下选项中正确的语句组是（ D ）

A)char s[];s="BOOK!";　　B)char *s;s={"BOOK!"};

C)char s[10];s="BOOK!";　　D)char *s;s="BOOK!";

【解析】字符数组名是地址常量，不能赋值给数组名，故 A、C 项错误，D 项指针 s 是个变量，将字符串常量赋值给 s，是正确的。B 项中给 s 赋值时不能用花括号将字符串括起来。

例 2：（2009-09-31）下面是有关 C 语言字符数组的描述，其中错误的是（D）

A)不可以用赋值语句给字符数组名赋字符串

B)可以用输入语句把字符串整体输入给字符数组

C)字符数组中的内容不一定是字符串

D)字符数组只能存放字符串

8.知识点：字符数组的输入与输出

例 1：（2009-09-13）有以下程序

```
#include <stdio.h>
main()
{char a[20]="How are you?",b[20];
scanf("%s",b);printf("%s %s\n",a,b);
}
```

程序运行时从键盘输入：How are you?<回车>

则输出结果为 How are you? How

【解析】scanf 函数输入，以空格作为字符串输入结束的标志，故字符数组 b 只接收到字符串"How"。

例 2：（2011-03）有以下程序

```
main()
{    char a[30],b[30];
     scanf("%s",a);
     gets(b);
     printf("%s\n %s\n",a,b);
}
```

程序运行时若输入：how are you? I am fine<回车>则输出结果是（B）

A)how are you?　　B)how

I am fine are you?I am fine

C)how are you? I am fine D)row are you?

【解析】：scanf 以空格作为字符串输入结束的标志，而 gets(s)不会。所以 a 得到的是“how”，b 得到的是剩下的“are you? I am fine”。故选择 B。

例 3：（2011-03）有以下程序

```
main()
{   char s[]="012xy\08s34f4w2";
    int i, n=0;
    for(i=0;s[i]!=0;i++)
        if(s[i]>='0'&&s[i]<='9')  n++;
    printf("%d\n",n);
}
```

程序运行后的输出结果是 (B)

A)0 B)3 C)7 D)8

【解析】：for 循环读取字符数组 s 中的字符，当读到 0 时退出循环，注意 0 和‘0’的区别，字符‘0’的 ASCII 码为 48，‘\0’的 ASCII 码才是 0，所以读取的字符为“012xy”中的字符，for 循环读取到字符后判断是不是数字字符，如果是 n 自增 1，“012xy”中的数字字符只有三个，所以选择 B。

9.知识点：字符串处理函数

例 1：(2011-09-32)有以下程序：

```
#include<stdio.h>
#include<string.h>
main()
{ char a[5][10]={“china”,”beijing”,”you”,”tiananmen”,”welcome”};
int i,j; char t[10];
for(i=0;i<4;i++)
  for(j=i+1;j<5;j++)
    if(strcmp(a[i],a[j])>0)
    { strcpy(t,a[i]);  strcpy(a[i],a[j]);  strcpy(a[i],t);}
 puts(a[3]);
}
```

程序运行后的输出结果是 (C)

A.beijing B.china C.welcome D.tiananmen

【解析】双层 for 循环部分是典型的排序方法，通过 strcmp 比较两个字符串的大小，然后借助 t 数组作为中转数组，通过 strcpy 函数完成 a[i]和 a[j]两个字符串的交换，使 a 数组按从小到大排序，即{”beijing” , “china” , ”tiananmen”,”welcome” ,”you” }

例 1：(2010-09-23)有以下程序

```
#include <stdio.h>
#include <string.h>
main()
{char x[]=”STRING”;
x[0]=0;x[1]=’\0’;x[2]=’0’;
printf(“%d %d\n”,sizeof(x),strlen(x));
}
```

程序运行后的输出结果是（B）

A)6 1 B)7 0 C)6 3 D)7 1

【解析】经过 x[0]=0;x[1]=’\0’;x[2]=’0’;后，x[]=”0\00ING”。strlen 测字符串的实际长度，不含字符串结束标志’\0’，故 strlen(x)为 0。sizeof 测字符串所占的存储空间，包含字符串结束标志，故 sizeof(x)为 7。

例 2：(2010-09-33)有以下程序

```
#include <stdio.h>
#include <string.h>
main()
{char a[20]=”ABCD\0EFG\0”,b[]=”IJK”;
strcat(a,b);   printf(“%s\n”,a);
}
```

程序运行后的输出结果是(B)

A)ABCDE\0FG\0IJK　　B)ABCDIJK　　C)IJK　　D)EFGIJK

【解析】把字符数组 b 中的字符串连接到字符数组 a 中字符串的后面，并删去字符串 a 后的第一个字符串结束标志’\0’。故输出结果为 ABCDIJK。

例 3：　(2010-09-31)下列选项中，能够满足“若字符串 s1 等于字符串 s2，则执行 ST”要求的是 (A)

A)if(strcmp(s2,s1)==0)ST;　　B)if(s1==s2)ST;

C)if(strcpy(s1,s2)==1)ST;　　D)if(s1-s2==0)ST;

【解析】若字符串 s1 等于字符串 s2，strcmp(s2,s1)==0，故答案是 A。

第四课 指针


指针
概念&定义
一个变量在内存空间的首地址称为该变量的指针
一般形式：类型名 * 指针变量名；
引用&运算
初始化：
类型名 *指针名=变量地址值（&）；
赋值运算：
int i,*p; p=&i;
只能在同一数据类型之间进行
算术运算
p±i 等价于 p ±i*d (i为整型数，d为p指向的变量所占字节数)
实质：指针的移动
关系运算
实质：判断同一数据集合中元素的位置关系
指针&数组
指针指向数组
一维数组
int a[6]; int *p=&a[0];
二维数组
int a[3][4]; int *p=&a[0];
指针引用数组元素
一维数组
a[i]等价于p[i] 等价于 *(p+i) 等价于*(a+i)
二维数组
int a[3][4]; int *p=&a[0][0]; 则p+i等价于&a[0][0]+i 指针逐个移动
行指针
定义形式
数据类型 (*指针名)[一维数组维数];
一维数组指针变量维数（指针）和二维数组列数（被指向的数组）必须相同
指针数组
定义形式
数据类型 *指针数组名[常量表达式]；
数组中的每个元素只能存放指针
指针与函数
指针变量做函数参数
传递的是地址
特点：共享内存,"双向"传递
指针型函数
定义形式
类型标识符 *函数名(参数表);
*表明返回值是一个指针（地址）
指向函数的指针
定义形式
数据类型 (*指针变量名)(函数参数表列);
函数调用形式
c=max(a,b); 等价于c=(*p)(a,b);
通过指针变量访问它所指向的函数
指向指针的指针
定义形式
类型名 **指针变量名；

本课重要知识点：

1.知识点：指针的概念与指针变量

- **指针就是地址。指针变量是用来存储地址**，而**一般变量是用来存储数值**。

例 1.(09-09-26)有如下程序输出结果为 1，2，2，1

```
#include <stdio.h>
main()
{   int m=1,n=2,*p=&m,*q=&n,*r;
    r=p ;p=q ;q=r ;
    printf("%d,%d,%d,%d\n",m,n,*p,*q) ;
}
```

【解析】p、q 为指针，初始化时 p 指向 m，q 指向 n。执行 r=p ;p=q ;q=r ;p 和 q 的值交换，从而 p 指向 n，q 指向 m。指针的改变不会应用 m、n 的值，最后输出*p 和*q 的值分别为 n、m 的值。

2.知识点：指针的引用

- 指针的引用是通过两个运算符实现**“&”和“*”**实现的

&是取地址符号，它的一半格式是：&变量名。例如：int i ; &i 表示取 i 的地址。

***是取值运算符**，它的一般格式：*指针变量名。例如:int i=10,*p=&i; *p 表示取 i 的值 10

例 1.(2011-09-25)若定义语句：int year=2009,*p=&year;,以下不能使变量 year 中的值增至 2010 的语句是 （D）

A.*p+=1; B.(*p)++; C.++(*p); D.*p++;

【解析】*和++同级别，按自左向右的结合方向，因此 D 选项可转变为*（p++），从而只是使指针发生移动，而不能将 p 所指变量增 1

例 2.(06—04—24)若有说明语句：double *p，a；则能通过 scanf 语句正确给输入项读入数据的程序段是 (D)

A)*p=&a； scanf("%lf"，p)；

B)*p=&a； scanf("%f"，p)；

C)p=&a； scanf("%lf"，*p)；

D)p=&a； scanf("%lf"，p)；

【解析】对于 scanf 函数，输入数据列表必须是合法地址表达式(可以使地址常量、指针)，A 选项、B 选项*使用错误。

例 3.(09-03-16)若有定义语句 doule x,y,*px,*py;执行了 px=&x;py=&y;之后，正确的输入语句是(C)

A) scanf("%f%f",x,y); B) scanf("%f%f"&x,&y);

C) scanf("%f%le",px,py); D) scanf("%lf%lf",x,y);

【解析】A,D 选项中的 x,y 缺少取地址符，B 项中&x 之前缺少逗号，格式不正确。

3.知识点：指针变量的初始化

- 指针变量在使用前必须要初始化，把一个具体的地址赋给它，否则引用时会出错，如果不指向任何数据就赋“空值”NULL。
- 指针变量两种初始化方法：方法一：int a=2，*p=&a；(定义的同时初始化)
  方法二：int a=2，*p；p=&a；(先定义后初始化)

例 1.(07—04—29)设已有定义：float x；则以下对指针变量 P 进行定义且赋初值的语句中正确的是 (D)

A)float *p=1034； B)int *p=(float)x； C)float p=&x； D)float *p=&x；

【解析】可以给一个指针赋值的只能是一个与该指针同类型的指针(或地址值)；故 A、B 选项错误；C 选项声明指针出错，P 前面的指针说明符*不能省略。

4.知识点：指针的运算

- ***p++和(*p)++之间的差别**：*p++是地址变化，(*p)++是指针变量所指的数据变化。一个指针变量加一个整数不是简单的数学相加，而是连续移动若干地址。当两个指针指向同一数组时，它们可以比较大小进行减法运算。

例如：int a[10],*p; p=a; p++;表示 p 移动一个存储单元，p 指向 a[1]，**只有当指着指向数组元素时，指针的运动才有意义。**

5:知识点：指针与数组

- 指针与一维数组

数组名代表数组的首地址。一维数组中，第一个元素的地址即为该数组的起始地址。建立指针变量与一维数组的联系：

例如： int a[6],*pa; pa=a;或 pa=&a[0];

说明：①数组名 a 代表该数组的首地址，也即 a[0]的地址。

② pa=a;或 pa=&a[0];使 pa 保存了数组 a 的首地址，pa,a,&a[0]都指向一个地址。

③以上操作可等价于 int a[6],*pa=a;

④如果 pa=&a[3]；表示*pa 的值就是 a[3]的地址。

注意：**数组 a 是一个地址常量，它永远指向数组的首地址，不能重新赋值。因此** a=&i;或 a++都是错误的。

例 1.(08—04—23)有以下程序

```
#include<stdio．h>
main()
{int a[]={1，2,3,4)，y，*p=&a[3]；
--p；y=*p；printf("y=%d\n",y)；
}
```

程序的运行结果是（D）

A)y=0 B)y=l C)y=2 D)y=3

【解析】程序中定义一个一维数组，并初始化，再一定义了一变量和一个指针(将指针指向数组下标为 3 的一个元素)，执行--p；(注意，当指针指一个元素后，指针变量加上“*”时，表示引用的是元素的值，当不加“*“时表示引用的是地址)，向前移动一个位置，指向 a[2]，y=+p；(将 p 指针指向的值 3 赋给变量 y)，输出 y 的值为 3。

例 2.(11-03-24)设有定义：double x[10],*p=x;，以下能给数组 x 下标为 6 的元素读入数据的正确语句是（ C ）

A)scanf("%f",&x[6]); B)scanf("%lf",*(x+6));

C)scanf("%lf",p+6); D)scanf("%lf",p[6]);

【解析】数组名是数组的首地址，p=x，指针 p 指向数组的首地址，要表示数组 x 下标为 6 的元素的地址可以有&x[6],x+6，p+6，&p[6],scanf 后面的参数列表必须是地址列表，B 中*（x+6）D 中 p[6]都是取数组 x 下标为 6 的元素的值，A 中格式控制符%f 与 double 类型不匹配，所以选 C。

6.知识点：用指针访问数组元素

- 通过指针引用数组元素

例如：int *p,a[5]; p=&a[0];

说明：①指针变量 p 指向了数组元素 a[0]，可以使用访问运算符“*”来引用变量 a[0];

例：*p=18;等价于 a[0]=18;

②*（p+1）表示 a[1] p+1 表示&a[1]

- 通过数组的首地址引用数组元素

例如：int a[5]；

说明：①a 是数组名，表示收地址，可以把 a 当做一个指针常量。

②*a 等价与 a[0]，*(a+1)等价与 a[1]；a 等价于&a[0]，a+1 等价与&a[1];

- 用带下标的指针变量引用一维数组元素

例如：int *p,a[5]; p=a;

说明：①p[0]表示 p 指针指向的内存单元，p[1]表示 p 指向的内存单元

②a[i]的等价引用方法：a[i]、p[i]、*(a+i)、*(p+i)

③a[i]的地址等价引用方法：&a[i]、&p[i]、a+i、p+i

例 1.(09-03—29)若有以下定义

int x[10]，*pt=x；

则对 x 数组元素的正确应用是_____B_____

A)*&x[10] B)*(x+3) C)*(pt+10) D)pt+3

【解析】引用数组元素时，注意取地址运算符&与指针运算符 t 的作用；指针运算符+用来取得指针变量所指存储空间的内容，取地址运算符&用来取得变量的地址值；A 选项数组下标越界；B 选项中+(x+3)等价于元素 X[3]；C 选项中。(pt+10)等价于 x[10]，数组下标越界；D 选项 pt+3 是元素 x[3]的地址，与&x[3]等价；故正确答案是 B。正确答案：B

例 1.(2011-09-27)有以下程序

```
#include <stdio.h>
void fun(int *p)
{printf("%d\n",p[5]);}
main()
{int a[10]={1,2,3,4,5,6,7,8,9,10};
 fun(&a[3]);
}
```

程序运行后的输出结果是 （D）

A.5 B.6 C.8 D.9

【解析】fun 函数被调用时将&a[3]通过传递付给了形参指针变量 p，此时可用*p 或 p[0]来表示 a[3]，因此 p[5]亦可表示 a[8]，所以输出结果为 9

7.知识点：指针与二维数组

- 任何一个二维数组均由若干个一维数组组成，a[0]、a[1]和 a[2]是一维数组名，数组名代表数组的首地址，因此 a[0]就代表数组元素 a[0][0]的地址，也即&a[0][0]。

注意：a 的值与 a[0]相同，但它们的基类型不同,a 可以等价于一个二维指针，而 a[0]是一维指针。因此 int a[3][4],*p=a；错误

取数组元素 a[i][j]的地址的几种方法：&a[i][j]；a[i]+j； *(a+i)+j；

- 指针数组的定义方式：

*指针数组名[常量表达式];如：int *p[3];

- 行指针的一般定义形式如下：

类型名 （*指针数组名）[常量表达式]； 如：int (*p)[2];

- **指针数组与行指针的区别**

1、int *p[3]；定义的是指针数组，表示一个数组，含有 3 个元素 p[0]、p[1]、p[2]，且这 3 个元素只能存放整型元素的地址

2、int (*p)[3];定义的是行指针，表示一个指针变量，它仅有一个存储空间，只能存放一个长度为 2 的一维数组指针。

例 1.(06—09—33)若有定义语句：int k[2][3]，*pk[3]；则以下语句正确的是(B)

A)pk=k； B)pk[0]=&k[1][2]； C)pk=k[0]； D)pk[1]=k;

【解析】题目中定义了一个二维数组和一个指针数组，pk 是指针数组名，不能被赋值，指针数组 pk 中的元素是指针，并且二维数组名是指向一维数组的指针常量，相当于行指针，二者不可转换，所以 A、C、D 选项均有误，本题答案选 B。

例 2.(09-09-27) 若有定义语句: int a[4][10],*p,*q[4];且 0<=i<4,则错误的赋值是 (A)

A)p=a B)q[i]=a[i] C)p=a[i] D)p=&a[2][1]

【解析】p 为基类型为 int 的指针，指向一个整形数据，也就可以指向一个数组元素，所以 D 正确。指针数组 q 的每个数组元素 q[i]的基类型也为 int，所以 p、a[i]、a[i]的基类型一致，选项 B、C 也是正确的。

8. 知识点：指针与函数

例 1.(08—04—40)设有定义语句 int(*f)(int);，则以下叙述正确的是__B___ ____。

A)f 是基类型为 int 的指针变量

B)f 是指向函数的指针变量，该函数具有一个 int 类型的形态

C)f 是指向 int 类型一维数组的指针变量

D)f 是函数名，该函数的返回值是其类型为 int 类型的地址

【解析】在 c 语言中，函数名代表此函数的入口地址，所因此，可以定义一种指向函数的指针来存放函数的入口地址，定义方式是：函数返回值类型(*函数指针名)(参数类型列表)；本题答案选 B。

9.知识点：指针与字符串

- 可以通过字符指针来处理字符串

例如：char *p="China"或者 char *p;p="China";把字符串赋值给指针 p，实质是把保存字符串"China"的那段内存的首地址赋值给指针 p，使得指针 p 指向了字符串，这样就可以通过指针来操作字符串了。

【注意】char str[10] ; **str="China";是错误的！数组名是地址常量，不能进行赋值操作！**

- **“三名主义”（考试的重点）**

数组名：表示第一个元素的地址。**数组名是地址常量名，不能进行赋值和自加(减)等运算。（考了很多次）**

函数名：表示该函数的入口地址。

字符串常量名：表示第一个字符的地址。

例 11.(2011-09-22)有以下程序（注：字符 a 的 ASCII 码值为 97）

```
#include <stdio.h>
main()
{ char *s={“abc”};
 do
{printf(“%d”,*s%10);++s;}
while(*s);
}
```

程序运行后的输出结果是 （B）

A. abc B. 789 C. 7890 D. 979898

例 11.(2011-09-30)若有定义语句：char *s1=”OK”;*s2=”ok”;,以下选项中，能够输出“OK”的语句是 （D）

A.if(strcmp(s1,s2)==0) puts(s1);

B.if(strcmp(s1,s2)!=0) puts(s2);

C.if(strcmp(s1,s2)==1) puts(s1);

D.if(strcmp(s1,s2)!=0) puts(s1);

【解析】strcmp 函数作为字符串比较函数，当 s1 等于 s2 所指字符串时结果为 0，当 s1 大于 s2 所指字符串时结果为 >0，当 s1 小于 s2 所指字符串时结果为 <0

例 11.(07—04—44)有下列程序：

```
main()
{char ch[]=“uvwxyz”, *pc;
```

pc=ch；printf(”％c＼n”，*(pc+5))；

｝程序运行后的输出结果是(A)

A)Z　　B)0　　C)元素 ch[5]的地址　D)字符 Y 的地址

【解析】指针 P 指向了字符数组 ch，*(pc+5)-->ch[5]-->’z’，printf 函数要求以％c 形式输出，故输出字符 Z。

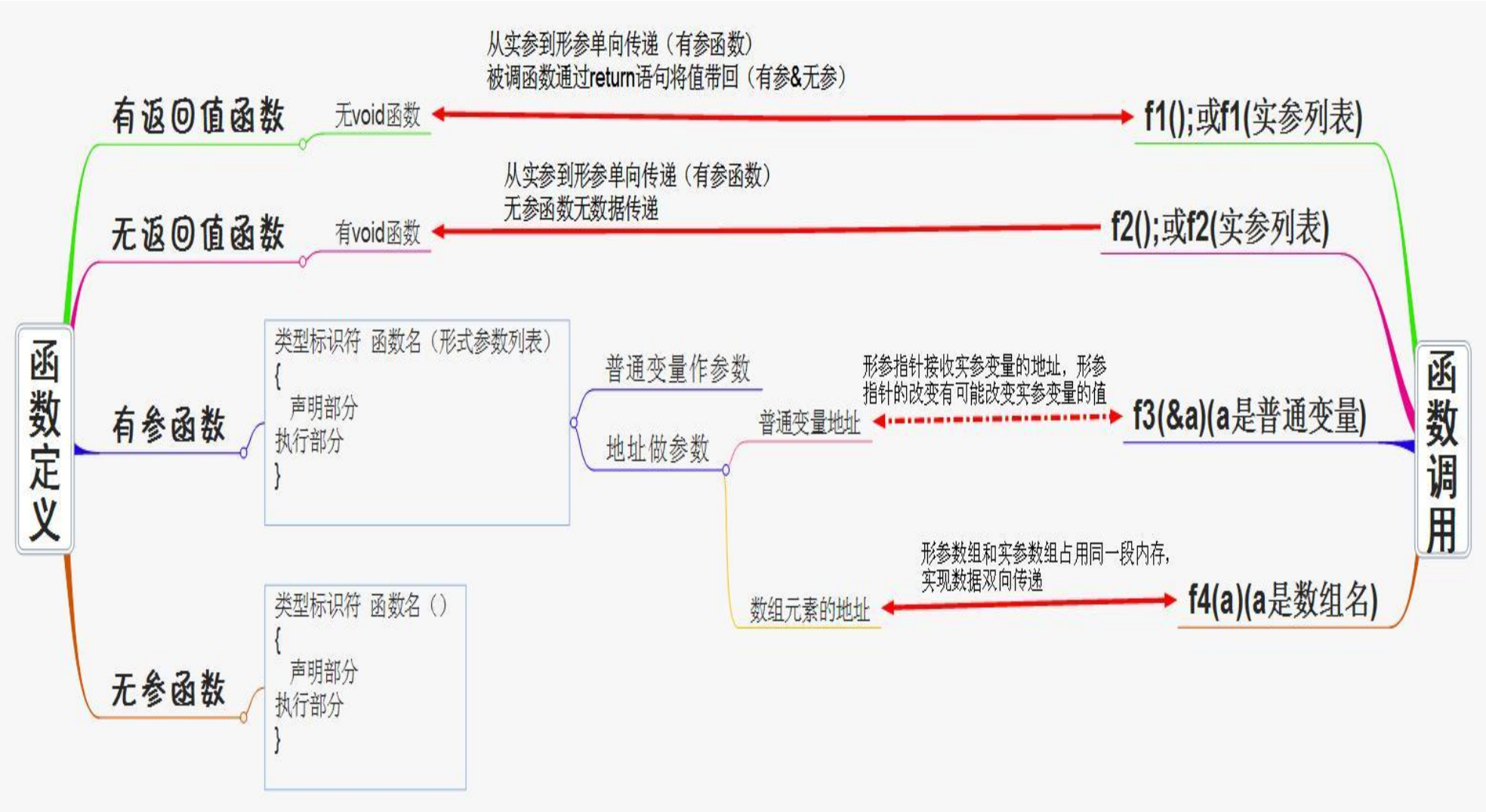
例 12.（09-09-37）设有定义：char *c;.以下选项中能够使字符型指针 c 正确指向一个字符串的是(A)

A)char str[]=”string”; c=str;　　B)scanf(“%s”,c);

C)c=getchar();　　D)*c=”string”;

【解析】选项 A 为正确用法。先将字符串存于字符数组中，然后将数组名赋给字符指针。选项 B 为无语法错误，但运行时可能会出现问题。原因是字符指针没有被赋值，是一个不确定的值，指向一个不确定的内存区域，这个区域可能存放有用的指令或数据。在这个不确定的区域重新存放输入的字符串，可能会发生无法预知的错误。选项 C 错误。getchar()函数输入一个字符给字符型变量，而不应该是字符指针。选项 D 错误。*c=”string”应该改为 c=”string”才是正确的。

第五课 函数


函数定义
有返回值函数
无void函数
从实参到形参单向传递（有参函数）
被调函数通过return语句将值带回（有参&无参）
f1();或f1(实参列表)
无返回值函数
有void函数
从实参到形参单向传递（有参函数）
无参函数无数据传递
f2();或f2(实参列表)
有参函数
类型标识符 函数名（形式参数列表）
{
声明部分
执行部分
}
普通变量作参数
地址做参数
普通变量地址
形参指针接收实参变量的地址，形参指针的改变有可能改变实参变量的值
f3(&a)(a是普通变量)
数组元素的地址
形参数组和实参数组占用同一段内存，实现数据双向传递
f4(a)(a是数组名)
无参函数
类型标识符 函数名（）
{
声明部分
执行部分
}
函数调用

本课重要知识点：

1、知识点：函数的概述

- C源程序是由**函数**组成的，函数由**函数首部与函数体**组成，函数体包括**声明部分和执行语句部分**组成。
- C语言程序可以由多个源程序组成，其中一个源程序文件包含main函数，其他函数可以写在另外的源程序文件中，**为单独文件**，并且**可以单独编译**。

例1：（10-03-11）以下叙述正确的是（ B ）

A)C语言程序是由过程和函数组成的

B)C语言函数可以嵌套调用，例如：fun(fun(x))

C)C语言函数不可以单独编译

D)C语言中除了main函数，其他函数不可以作为单独文件形式存在

【解析】选项A错误，C语言中没有过程的概念；选项B正确，C语言可以嵌套调用（此知识点在后面介绍）；选项C错误，C语言中可以对包含一个或多个函数的源程序单独编译；选项D错误，C语言函数可以单独存在。

- C程序的执行总是**从main函数开始**，完成对其它函数的调用后再返回到main函数，最后**由main函数结束**整个程序。main函数可以调用其它函数，而**不允许被其它函数调用**。**一个C源程序必须有，也只能有一个主函数main。**

例1：下列叙述中正确的是（ C ）

A)C语言程序将从源程序中第一个函数开始执行

B)可以在程序中由用户指定任意一个函数作为主函数，程序将从此开始执行

C)C语言规定必须用main作为主函数名，程序将从此开始执行，在此结束

D)main可作为用户标识符，用以命名任意一个函数作为主函数。

- 在一个函数的函数体内，不能再定义另一个函数，即**不能嵌套定义**。

例1、下列叙述中正确的是（ D ）

A)每个C程序文件中都必须要有一个main()函数

B)在C程序中main ()函数的位置是固定的

C)C程序中所有函数之间都可以是固定的

D)在C程序的函数中不能定义另一个函数

【解析】一个C源程序可以由多个C程序文件，但只能有一个main()函数，并不是每个C程序文件都必须要有一个main()，main()函数可以放在任意位置，且不允许被其他函数调用，因此正确答案是D。

2、知识点：函数的定义

例1：以下正确的函数定义形式是：（C）

A)double fun(int x,int y);{}　　B） double fun(int x ;int y){}

C) double fun(int x,int y){}　　D) double fun(int x, y);{}

【解析】在函数中对形参的说明，要求对每个形参都必须说明类型，各参数之间用“,”隔开，函数定义后应无“;”。

- 函数返回return语句的一般形式：return 表达式；或return (表达式);

在函数中允许有多个return语句，但每次只能有一个return语句被执行。

例1：（10-03-24）以下关于return语句叙述中正确的是（ B）

A）一个自定义函数中必须有一条return语句

B）一个自定义函数中可以根据不同情况设置多条return语句

C）定义void类型的函数中可以有带返回值的return语句

D）没有 return 语句的自定义函数在执行结束时不能返回到调用处。

【解析】自定义函数中可以没有 return 语句（如不需要返回值的函数，常见 void 类型））,可以有多条 return 语句（如在分支结构中，可以从不同分支中返回到调用程序，常见 if..else）；定义成 void 类型的函数，不允许从该函数取得返回值，也不允许使用 return 语句。没有 return 语句的函数在执行到函数的最后一条语句后自动返回达到调用处。

➢ 函数值的类型与函数定义中函数的类型保持一致，如不一致，**以函数类型为准**。

例 2：(06-04-23)已定义以下函数

int fun(int *p){return *p；} fun 函数返回值是（ B ）

A)不确定的值 B)一个整数 C)形参 P 中存放的值 D)形参 P 的地址值

【解析】函数返回性类型取决于定义函数时函数名称前的类型，fun 前是 int，表明函数 fun 执行完毕后返回一个整型数。正确答案：B

➢ return 后的表达式可以是常量、变量、表达式，也可以空。

如：return ; return 0; return a; return (a+b);

3、知识点：形参和实参及函数调用

- **形参出现在函数定义中（**即定义函数时函数名后面括号中的参数），**实参出现在主调函数中**（调用函数时函数名后面括号中的参数）
- **形参在本函数体内都可以使用**，离开该函数则不能使用。**实参在主调函数中内有效，**进入被调函数后，实参变量也不能使用。（注：在进行函数调用时，函数必须有确定的值）
- **实参向形参单向传递数值**，不能将形参的值反向的传送给实参。实参和形参在数据上、类型上、顺序上要一致。在函数调用过程中，**形参的变化不会改变实参的变化。**

例 1、（09-09-24）有以下程序

```
#include <stdio.h>
void fun(int p)
{   int d=2;        p=d++; printf("%d",p);  }
main()
{   int a=1;
    fun(a); printf("%d\n",a);
}
```

程序运行后的输出结果是（ C ）

A) 32 B)12 C)21 D)22

【解析】此题考点为函数参数的传递。C 语言中函数参数的传递是值传递，是把实参的值传给形参，是单向传递，形参的改变不会影响到实参的值。程序中，把实参 a 的值传给形参 p，p=1，然后 p=d++，再次赋值后 p=2，输出 p 的值 2。返回到主程序中，输出 a 的值为 1。（形参 p 的改变不会影响到实参 a 的值，a 的值仍为 1）. 因此答案为 C.

- 在传递数值时，**形参和实在分别占据不同的存储单元**（形参变量只有在被调用时才分配内存单元，调用结束后，即刻释放分配的内存单元）

例 1、(07-04-25)若函数调用时的实参为变量时，以下关于函数形参和实参的叙述中正确的是（ D）

A)函数的实参和其对应的形参共占同一存储单元

B)形参只是形式上的存在，不占用具体存储单元

C)同名的实参和形参占同一存储单元

D)函数的形参和实参分别占用不同的存储单元

【解析】本题考点是函数的参数定义，在函数中，主调数中里边的参数叫做实参，而被调函数里边的参数叫做形参，当函数传递时，传递的是值，它们占用的不

是同一存储单元，而且在传递中，是单向传的，由实参传向形参。

- **“传值”与“传址”的区别：**传数值的话，形参的变化不会改变实参的变化。传地址的话，形参的变化就有可能改变实参所对应的量。

例 1：（10-03-26）有以下程序

```
#include <stdio.h>
void fun(char *c,int d)
{    *c=*c+1;    d=d+1;
     printf("%c,%c",*c,d);
}
main()
{ char b='a', a='A';
  fun(&b,a);
  printf(("%c,%c\n",b,a);
}
```

程序运行后的输出结果是(A)

A) b,B,b,A　　B)b,B,B,A　　C)a,B,B,a　　D)a,B,a,B

4、知识点：函数调用的一般形式和调用方式

- 函数调用的一般形式为：**函数名（实际参数表）**;

实参可以是常数、变量或其他构造类型数据及表达式，也可以没有（当为无参函数时）

- 函数调用的方式：

（1）**函数表达式**：函数作为表达式中的一项出现在表达式中，以函数返回值参与表达式运算。例如：z=max(x,y);

（2）**函数语句**：函数调用的一般形式上加上分号。例如：printf("%d\n",a);

（3）**函数实参**：函数作为另一个函数调用的实际参数出现。例如：printf("%d",max(x,y));

- 程序从 main 函数进入，上往下执行，当碰到函数名后，把值传给调用函数，当程序得到了返回值或调用函数结束，再顺序往下执行，最后到 main 函数结束。

5、知识点：函数的声明及其位置

- 函数要“先定义后调用”，或“先声明再调用后定义”。函数的声明**一定要**有函数名、函数返回值类型、函数参数类型，但**不一定要**有形参的名称。

一般形式为：

类型说明符 被调函数名(类型 形参，类型 形参…)；

或　　类型说明符 被调函数名(类型，类型…)；

注意：其末尾“；”不能省

例如：int max(int a,int b);　　或　　int max(int ,int);　两者功能相同

例 1、(06-09-32)若有以下函数首部

int fun(double x[10]，int *n)

则下面针对此函数声母语句中正确的是（D）

A)int fun(double x，int *n)；　　B)int fun(double，int)；

C)int fun(double*x，int n)；　　D)int fun(double *，int。)；

【解析】本题考点是对函数的声明，在函数声明的时候,其格式如下：返回值类型函数名(参数类型[参数名]，参数类型[参数名]……)其中，参数名可以不写，但是参数类型一定要写全

- **省去主调函数中对被调函数的函数说明**的几种情况：

（1）被调函数的返回值是整型或字符型时（返回值自动按整型处理）

（2）当被调函数的函数定义在主调函数之前时

（3）在所有函数定义之前，在函数外预先说明了各个函数的类型

例 2：（10-09-24）有以下程序

```
#include <stdio.h>
int f(int x);
main()
{ int n=1,m;
  m=f(f(f(n)));    printf("%d\n",m);
}
int f(int x)
{ return x*2;}
```

程序运行的输出结果(　D　)

A) 1　　B)2　　C)4　　D)8

【解析】f 函数定义在 main 下面，但在外部进行声明 int f(int x);因此在 main 中可直接调用。

三次调用函数:f(n)=f(1)=2　f(2)=4　f(4)=8 m=f(f(f(n)))=f(f(f(1)))=f(f(2))=f(4)=8

例 3：（11-03-12）有以下程序，请在 【12】 处填写正确语句，使程序可正常编译运行。

```
【12】;
main()
{    double x,y,(*p)();
     scanf("%lf%lf",&x,&y);
     p=avg;
     printf("%f\n",(*p)(x,y));
}
double avg(double a,double b)
{ return((a+b)/2);}
```

答案：double avg(double a,double b); 或 double avg(double ,double);

【解析】当函数定义在调用之后，调用前必须对函数进行申明，申明的形式和函数定义中的函数头部分相同，最后加分号即可。

6、知识点：函数的嵌套调用

- C 语言中不允许作嵌套的函数定义，但是允许在一个函数的断定仪中出现对另一个函数的调用，即嵌套调用。

例 1：（06-04-33）有以下程序

```
 int funl(double a){return    a*=a；}
 int fun2(double x，double y)
{double a=0，b=0；
 a=funl(x)；b=funl(y)；return (int)(a+b)；
}
main()
{double    w；w=fun2(1.1,2.0)；……}
```

程序执行后变量 w 中的值是(C)

A)5.2l　　B)5　　C)5.0　D)0.0

【解析】此题考查函数的嵌套调用。main 函数中调用了 fun2 函数；fun2 函数中又调用了如 fun1 函数 w=fun2(1.1,2.0)-->w=(int)(funl(1.1)+funl(2.0))→w=(int)5.21；w 值为 5，但是 w 是 double 类型变量，故将 5 转换成 5．0 赋值给变量 w

例 2、（09-3-24）有以下程序

```
#include<stdio.h>
int f(int x,int y)
```

```
{return ((y-x)*x);}
main()
{    int a=3,b=4,c=5,d;
     d=f(f(a,b),f(a,c));
          printf("%d\n",d);
     }
```

程序运行后的输出结果是（ B ）

A) 10 B)9 C)8 D)7

【解析】程序的 main()函数中，定义了 4 个变量，前 3 个分别初始化，D 用来接收调用函数后返回的值，主函数一共调用了 3 次 f()函数，第一次传递 a,b 的值，给形参 x,y，返回（y-x）*x 的值为 3，第二次将 a,c 的值传递给形参 x,y，返回（y-x）*x 的值为 6，接着将 f（a,b），f(a,c)的值 3、6 传递给形参,执行后返回值 9，赋值变量 d，最后输出结果为 9

7、知识点：函数的递归调用

- **函数直接或间接地调用自己称为函数的递归调用**。递归调用必须有一个明确的结束递归的条件。在做递归题时可把递归的步骤一步步写下来，不要弄颠倒了。

例 1、(04-09-37)在函数调用过程中，如果函数 funA 调用了函数 funB，函数 funB 又调用了函数 funA，则 （ B ）

A)称为函数的直接递归调用

B)称为函数的间接递归调用

C)称为函数的循环调用

D)C 语言中不允许这样的递归调用

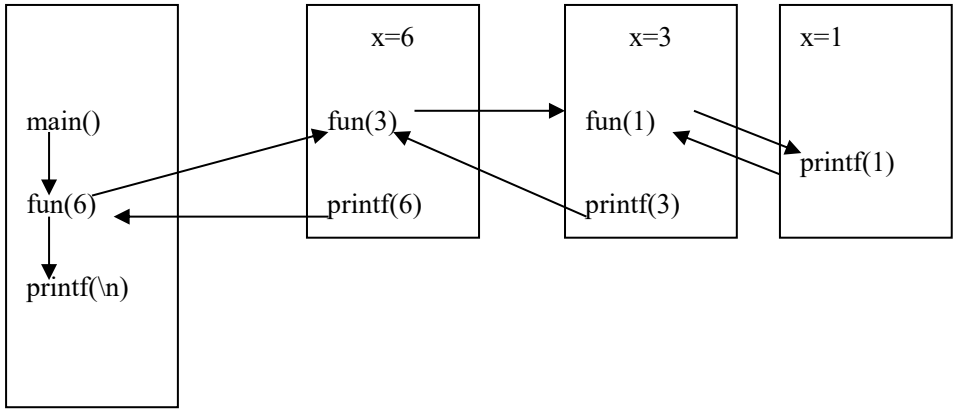
【解析】此题考查函数递归的基本概念。一个函数直接或间接调用自己称为函数的递归调用；若直接调用了自己，则称为直接递归调用，若间接调用了自己则称为间接递归调用；根据间接递归调用的概念可知，本题中的函数调用为间接递归调用。

例 2、（09-09-15）

```
#include <stdio.h>
fun(int x)
{    if(x/2>0) fun(x/2);
     printf("%d  ",x);
}
main()
{    fun(6);    printf("\n");}
```

程序运行后的输出结果是__1 3 6____

【解析】程序的执行过程是先递推、后递归的过程。当 x=1 时，条件不成立，递推终止。程序执行流程参照示意图。



例 3：（10-03-33）有以下程序

```
#include <stdio.h>
```

```
int f(int t[],int n);
main()
{ int a[4]={1,2,3,4},s;
  s=f(a,4); printf("%d\n",s);
}
int f(int t[],int n)
{     if(n>0) return t=[n-1]+f(t,n-1);
      else return 0;
}
```

程序运行后的输出结果是（ B）

A) 4　　B) 10　　C)14　　D)6

【解析】 s=f(a,4)=t[3]+f(a,3)=4+f(a,3)=4+t[2]+f(a,2)=4+3+f(a,2)=7+t[1]+f(a,1)=7+2+f(a,1)=9+t[0]+f(a,0)=9+1+f(a,0)=10+0=10

例 4：（11-03-32）设有如下函数定义

```
int fun(int k)
{     if (k<1) return 0;
      else if(k==1) return 1;
      else return fun(k-1)+1;
}
```

若执行调用语句：n=fun(3);，则函数 fun 总共被调用的次数是(B)

A)2　　B)3　　C)4　　D)5

【解析】递归调用，fun(3)—> fun(2)—> fun(1) ;fun(1)—1—> fun(2)—2—> fun(3)—3—>n；所以 fun 总共被调用的次数是 3，最后 n 为 3。

例 5：（11-03-34）有以下程序

```
int fun (int x,int y)
{     if (x!=y) return ((x+y)/2);
     else return (x);
}
main()
{     int a=4,b=5,c=6;
      printf("%d\n",fun(2*a,fun(b,c)));
}
```

程序运行后的输出结果是(B)

A)3　　B)6　　C)8　　D)12

【解析】函数递归调用，fun(8,fun(5,6)),其中 fun(5,6)因为 5!=6, return ((x+y)/2)为 5，所以 fun(8,fun(5,6))为 fun(8,5)，8!=5, return ((x+y)/2)为 6

8、知识点：局部变量和全局变量

- 在一个**函数内部或复合语句内部定义**的变量，它**只在本函数范围内有效**，也就是说只有在本函数内才能使用它们，在函数以外是不能使用这些变量的。这**称为局部变量**，局部变量又称为内部变量。函数的形参也属于局部变量。
- **在函数外部定义**的变量，称为**全局变量**，又称为外部变量。全局变量可**以被本文件的其他函数所共用**。

例 1、(07-09-40)在一个 C 语言源程序文件中所定义的全局变量，其作用域为（D）

A)所在文件的全部范围　　B)所在程序的全部范围

C)所在函数的全部范围　　D)由具体定义位置和 extem 说明来决定范围

【解析】本题考点是全局变量的作用域，全局变量的作用域是，从变量定义开始，一直到程序程结束为止，当然，也可以在其它文件中用这个以定义的全局变是，但是要通过 extern 来进行声明，本题答案选 D。

例 2、(09-09-11)有以下程序

```
#include<stdio.h>
int a=5;
void fun(int b)
{     int a=10; a+=b;
      printf("%d",a)
}
main()
```

```
{    int c=20;
     fun(c); a+=c;
     printf("%d\n",a);
}
```

程序运行后的输出结果是____3025____

【解析】全局变量的作用域为定义开始到文件结束。局部变量的作用域为定义其函数内部。当局部变量与全部变量同名时，在局部变量作用域内，全部变量不起作用。

执行函数 fun(c): 实参 c 将值传给形参 b，即 b=20，函数内部定义局部变量 a 起作用,a=10,a+=b a=a+b=10+20=30，输出 a 的值为 30

返回到主函数： a+=c 此时 a 为全局变量的值 a=a+c=5+20=25 输出 a 值 25

注意格式控制符无空格，所以连续输出

9 知识点：变量存储类别（auto 变量、register 变量、static 变量）

- **静态变量（static）：**

（1）**在编译时分配存储空间，所占存储单元直到程序结束时才释放**，它的值在程序运行过程中一直存在，且变量的初始化只进行一次。

（2）**static 说明符可以用于全局变量，也可用于局部变量**（auto 和 resgiter 不可以定义全局变量）但是，当它说明变量后，只要这个变量还在程序中存在，那么不管用到还是没有用到，它都将会占用内存单元。

例 1：（10-09-36）有以下程序：

```
#include <stdio.h>
int f()
{    static int x=1;
     x+=1;     return x;
}
main()
{    int i,s=1;
     for(i=1;i<=5;i++) s+=fun();
     printf("%d\n",s);
}
```

程序运行后的输出结果是(B)

A)11 B)21 C)6 D)120

【解析】fun 函数被调用了 5 次，每次调用后的值返回值累加到 s 上。

第一次：静态局部变量定义有效，x=2,返回值为 2。s=s+fun()=1+2=3;从第 2 次调用开始，静态局部变量的定义相当于不存在，直接使用 x 的值。

第二次：x=3 s=3+3=6 第三次：x=4 s=6+4=10

第四次：x=5 s=10+5=15 第五次：x=6 s=15+6=21

例 2：（11-03-34）有以下程序

```
int fun()
{    static int x=1;
     x*=2;
     return x;
}
main()
{    int i,s=1;
     for(i=1;i<=3;i++) s*=fun();
     printf("%d\n",s);
}
```

程序运行后的输出结果是**（D）**

A)0 B)10 C)30 D)64

【解析】静态变量 static 类型，在函数执行完后空间并不释放，而且只初始化一次。所以第一次调用后，x 的值为 2，返回后 s 的值为 2，第二次调用后，x 的值为 2*2=4，返回后 s 的值为 2*4=8，第三次调用后，x 的值为 4*2=8，返回后 s 的值为 8*8=64，故选 D。

➢ **自动变量（auto）**

（1）**自动变量的存储空间是当程序执行到定义它的函数或语句块时才分配，当函数执行结束后自动释放**，变量的值也就不再存在。

（2）局部变量的存储类别**默认为 auto**，可以省略不写，但它不能定义全局变量。

➢ **寄存器变量（register）：**

例 1、(08-04-39)在 C 语言中，只有在使用时才占用内存单元的变量，其存储类型是（ A ）。

A)auto 和 register　B)extern 和 register　C)auto 和 static　D)static 和 register

【解析】auto 变量在进入函数体时，系统为其分配存储单元，退出时自动释放；register 是建议编译程序将变量的值保存在 cpu 的寄存器中，而不是占用内存单元；extern 可以对全局变量加以说明，扩展其作用域，在整个程序中都有效，所以会一直占用内存单元；static 说明符可以用于全局变量，也可用于局部变量，使变量在整个程序执行期间一直占用内存单元，本题答案选 A。

例 2：（10-09-34）设函数中有整形变量 n，为保证其在未赋初值的情况下初值为 0，应该选择的存储类别是（ C ）

A）auto　B)register　C)static　D)auto 或 register

【解析】对于静态局部变量(static)，如在定义时不赋初值的话，编译时自动赋初值 0（对数值型变量）或空字符（对字符变量）。而对于自动变量（auto 或者缺省不写），如果不赋初值则它的值是一个不正确的值。

10、内部函数与外部函数

- **用 static 进行说明的函数称为静态函数，也叫内部函数**。静态函数只能被本编译单元体内的函数调用。一般形式：static 类型标识符 函数名（形参表） 例如：static int fun(){…}

11、知识点：库函数

- 函数可以分为库函数和用户自定义函数。库函数由 C 系统提供，用户无须定义，在**调用 C 语言标准库函数时要包含 include 命令**，include 命令行**以#开头**；

例 1、(07-04-17)若有代数式 $\sqrt{|n^x+e^x|}$ (其中 e 仅代表自然对数的底数，不是变量)，则以下能够正确表示该代数式的 C 语言表达式是（ C）

A)sqrt(abs(n^x+e^x))　B) sqrt(fbs(pow(n,x)+pow(x,e)))

C)sqrt(fabs(pow(n,x)+exp(x,e)))　D)sqrt(fabs(pow(x,n)+exp(x)))

【解析】exp()函数的功能是返回以自然数 e 为底、函数参数 x 为幂的指数值 e^x ;pow(n,x)函数是计算 n^x ；fabs()函数的功能是返回函数参数的绝对值；sqrt()用于返回函数参数的平方根。所以正确答案为 C。

例 2：以下说法不正确的是（B）

A)标准库函数按分类在不同的头文件中声明

B)用户可以重新定义标准库函数

C)系统不允许用户重新定义标准库函数

D)用户若需要调用标准库函数，调用前必须使用预编译命令将该函数所在文件包括到用户源文件中

【解析】对每一类库函数，都有相应的头文件名，调用某个库函数时，用户在源程序中须用 include 命令包含其头文件名。每一类标准库函数是 C 语言本身提供的，用户不能重新定义标准库函数。

12、知识点：数组与函数

- 数组用作函数参数有两种形式：把**数组元素（下标变量）作为实参**使用和把**数组名**作为函数的形参和实参使用。

- **数组元素作函数的实参**与普通变量并无区别，所进行的传递是**值**，**形参变量和实参变量占据由编译系统分配的两个不同的内存单元**；
- **数组名作为函数参数**时，所进行的传递是**地址**（相当于把实参数组的首地址赋予形参数组名或指针名），**形参数组与实参数组为同一数组，共同拥有一段内存单元**；

例 1：（2011-09-31）以下程序的主函数中调用了在其前面定义的 fun 函数

```
#include<stdio.h>
main()
{double a[15],k;
k=fun(a);
}
```

则以下选项中错误的 fun 函数首部是 （D）

A.double fun(double a[15]) B.double fun(double *a)

C.double fun(double a[]) D.double fun(double a)

【解析】当一维数组名 a 作为实参时，对应形参有三种表示形式：*a、a[ ]、a[N]（注：N 为定义 a 数组时的元素个数）

例 1：（11-03-11）已知 a 所指的数组中有 N 个元素。函数 fun 的功能是，将下标 k(k>0)开始的后续元素全部向前移动一个位置。请填空。

```
Void fun(int a[N],int k)
{   int i;
    for(i=k;i<N;i++) a[【11】] = a[i];
}
```

答案：i-1

【解析】a[i]的值赋给 a[i-1]

例 2：（10-09-32）有以下程序

```
#include <stdio.h>
#define N 8
void fun(int *x,int i)
{   *x=*(x+i); }
main()
{   int a[N]={1,2,3,4,5,6,7,8},i;
    fun(a,2);
    for(i=0;i<N/2;i++)
    { printf("%d",a[i]);}
    printf("\n");
}
```

程序运行后的输出结果是（ C ）

A) 1 3 1 3 B)2 2 3 4 C) 3 2 3 4 D) 1 2 3 4

【解析】数组名代表数组首地址，即 a[0]的地址。函数调用：fun(a,2) 参数传递后，形参指针 x 获得数组首地址，即指向 a[0]，形参变量 i=2;*x 即为 a[0]，*(x+2)即 a[2]，*x=*(x+2)相当于 a[0]=a[2]，a[0]变为 3，a[2]不变。返回到主程序：循环 4 次，输出 a 数组前 4 个元素： 3 2 3 4

例 3：（10-09-25）有以下程序

```
#include <stdio.h>
void fun(char *s)
{  while (*s)
     {if (*s%2==0) printf("%c",*s);   s++;}
}
main()
{   char a[]={"good"};
```

fun(a);printf("\n");

}注意：字母 a 的 ASCII 码值为 97，程序运行后的输出结果是（ A ）

A) d B)go C)god D)good

【解析】调用 fun(a)函数，实参字符数组名 a 传递给形参字符指针 s，使 s 指向字符串“good”；在函数 fun 中对字符串进行遍历，如果指针 s 指向字符 ASCII 码值是偶数则输出该字符。在字符串“good”中，只有“d”的 ASCII 码值为偶数

13 指针与函数

- **函数指针变量是指向函数的指针变量**。

  定义的一般形式为: 类型说明符 （*指针变量名）(); 例如: int (*pf)();

  调用函数的一般形式：（*指针变量名）（实参表） 例如：z=(*p)(x,y);

  ↳注: （*指针变量名）中的“()”不能少

例 1：（09-09-33）设有以下函数：

void fun(int n,char *s){…} 则下面对函数指针的定义和赋值均正确的是（ A ）

A)void (*pf)(); pf=fun; B)void *pf(); pf=fun;

C)void *pf(); *pf=fun; D)void (*pf)(int,char);pf=&fun;

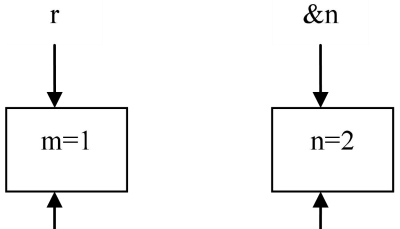
【解析】本题考点为指向函数的指针的用法。函数名代表函数的入口地址，可以赋值给指向函数的指针。指向函数的指针应该定义为 void （*pf）().如定义为 void *pf()，则表示函数 pf 返回值为一个基类型为 void 的指针。

- **函数指针变量不能进行算术运算**。
- 指针型函数是指**返回指针值**的函数。定义的一般形式为：

  类型说明符 ***函数名**（形参表）{…} 例如：int *ap(int x,int y){}

例 1：(10-09-11)有以下程序

#include <stdio.h >



int *f(int *p,int *q)

main()

{ int m=1,n=2,*r=&m;

r=f(r,&n) ; printf("%d\n",*r) ;

}

int *f(int *p,int *q)

{ return (*p>*q) ?p :q ;}

程序运行后的输出结果是_____2_____

【解析】本题中 f 是个指针型函数，返回值为指针值。如示意图。主程序中，指针 r 指向 m，函数调用 f(r,&n)时，实参传递给形参，形参指针 p 指向 m，指针 q 指向 n，*p 的值为 m 的值 1，*q 的值为 n 的值 2，所以函数返回值为 q，返回给主程序中的指针 r，即指针 r 指向了 n，最后输出*r 的值为 n 的值 2.

例 2：(11-03-29)有以下程序

```
void fun(char *c)
{   while(*c)
    {     if(*c>='a'&&*c<='z') *c=*c-('a'-'A');
              c++;
    }
}
main()
{   char s[81];
    gets(s); fun(s); puts(s);
```

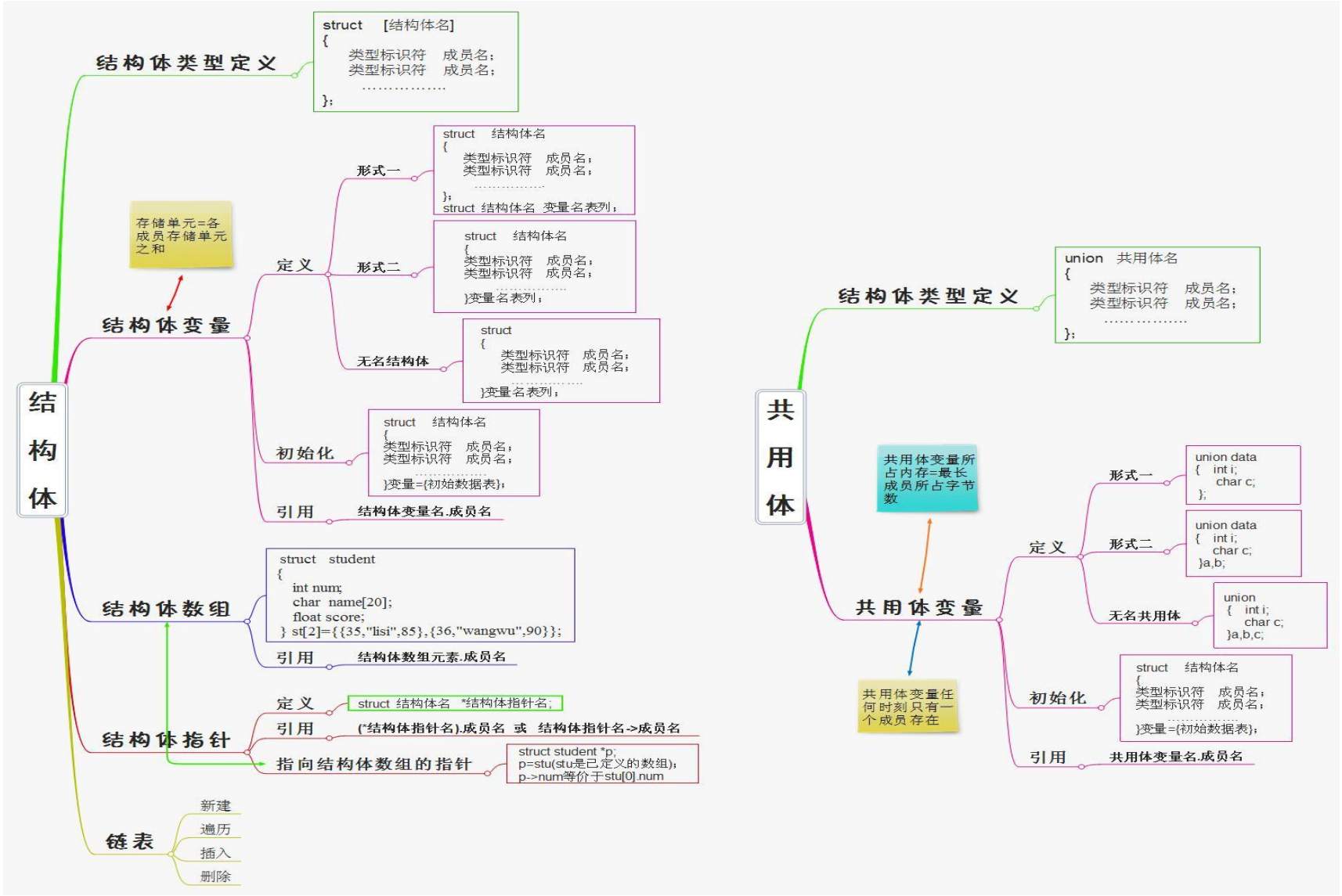
}当执行程序时从键盘上输入 Hello Beijing<回车>，则程序的输出结果是**(C)**

A)hello beijing B)Hello Beijing C)HELLO BEIJING D)hELLO Beijing

【解析】gets(s)将输入的 Hello Beijing 赋给了字符数组 s，调用 fun(s)，将字符数组 s 的首地址传给了 c，c 指向的即为字符数组的首地址，通过指针变量 c 取得字符数

组中的字符判断是不是小写字母，如果是变成大写字母。*c=*c-(‘a’-‘A’)完成小写换大写的功能。故选 C。另外注意 gets(s)和 scanf(“%s”,s);的区别：scanf 以空格作为字符串输入结束的标志，而 gets(s)不需要

第五课 结构体、链表与共用体



本课重要知识点：

1.知识点：用 typedef 定义类型

- 功能：为已有数据类型取别名
- 类型定义形式： **typedef 旧类型名 新类型名；（例如：typedef double D ； ）**

注意：

（1） 定义的新名只是原名的一个**别名**，并不是建立一个新的数据类型；

（2） 用新名和原名定义的对象具有**相同的性质和效果**。

例 1：（2010-09-39）若有以下语句

typedef struct S

{int g; char h;} T;

以下叙述中正确的是 （ **B** ）

A）可用 S 定义结构体变量

B）可用 T 定义结构体变量

C）S 是 struct 类型的变量

D）T 是 struct S 类型的变量

【解析】此题中用 typedef 说明的新类型名 T，与中间部分的结构体类型具有相同的性质和效果

3. 知识点：结构体变量的定义

	具体形式	实例	说明

形式1	struct 结构体名
	{
	类型标识符 成员名;
	类型标识符 成员名;
	……
	};
	struct 结构体名 变量名表列;
		struct student
		{ char num[20];
		char name[20];
		char sex;
		float score[4];
		};
		struct student stu1, stu2;
			特点：先定义类型，后定义变量
			注意：定义变量时，必须以 struct student 开头，缺一不可

形式2	struct 结构体名 { 类型标识符 成员名; 类型标识符 成员名; …… }变量名表列;	struct student { char num[20]; char name[20]; char sex; float score[4]; } stu1, stu2;	**特点：**定义类型的同时定义变量 **注意：之后再**定义同类型变量时，也必须必须以 struct student 开头
形式3	struct { 类型标识符 成员名; 类型标识符 成员名; …… }变量名表列;	struct { char num[20]; char name[20]; char sex; float score[4]; } stu1, stu2;	**特点：**定义类型的同时定义变量 **注意：此形式省略了结构体名，预示着后续程序中将不再定义该类型变量**
形式4	typedef struct 结构体名 { 类型标识符 成员名; 类型标识符 成员名; …… } STU; STU 变量名表列;	typedef struct student { char num[20]; char name[20]; char sex; float score[4]; } STU; STU stu1, stu2;	**特点：**先定义类型，后定义变量 **注意：用 STU 或** struct student 定义变量具有同等效果，用 STU 形式简洁方便

例 1：(:2009-09-36)下面结构体的定义语句中，错误的是 （ B ）

A)struct ord {int x;int y;int z;}; struct ord a;

B)struct ord {int x;int y;int z;} struct ord a;

C)struct ord {int x;int y;int z;} n;

D)struct {int x;int y;int z;} a;

【解析】**遵循先定义类型，再定义该类型变量的原则，类型定义末尾的分号“;”必不可少**

2.知识点： 结构体变量各成员的引用

- 引用结构体变量的成员的方法是：**结构体变量名.成员名** （**例如：stu1.sex**）
- 结构体指针变量引用成员的方法是：

形式 1：**（*指针变量名）.成员名** （**例如：（*p）.num**）

形式 2： **指针变量名->成员名** （**例如：p->num**）

- 运算符级别，**注意：（）和->和 . 同级别，均大于***

设有：

```
struct student
{ char   num;
  char   name[20];
  struct date {int year,mon,day} birth;
  float score[4];
};
```

struct student *p，stu1；

p=&stu1；

成员类型	成员	操作方式	说明
普通变量	例如:引用成员变量 num	**直接赋值：stu1.num=201101;** **间接赋值：p->num=201101** **输入：scanf(“%d”,& stu1.num);** **输出：printf(“%d”,stu1.num);**	
数组	例如:引用成员数组元素 score[0]	**赋值：stu1.score[0]=75;** **间接赋值：p-> score[0]=75;** **输入：scanf(“%f”,& stu1. score[0]);** **输出：printf(“%f”,stu1. score[0]);**	
字符串	例如:字符数组成员作为字符串 name	**赋值：strcpy(stu1.name,”zhangsan”);** **间接赋值：strcpy(p->.name,”zhangsan”);** **输入：scanf(“%s”, stu1.name);** **输出：printf(“%s”,stu1.name);**	%s 作为格式控制时，scanf 和 printf 的输入和输出项均为地址
结构体嵌套	例如:引用内嵌结构体变量 birth 的成员 year	**赋值：stu1.birth.year=1989;** **间接赋值：p-> birth.year=1989;** **输入：scanf(“%d”,& stu1. birth.year)** **输出：printf(“%d”,stu1. birth.year)**	

例 1：(2009-9-12)设有定义

```
struct  person
{  int  ID;char  name[12];}  p;
```

请将 scanf("%d",_____);语句补充完整,使其能够为结构体变量 p 的成员 ID 正确读入数据。

【解析】此题答案是 **&p.ID** ，即在成员 ID 前缀 p. 表示其从属于变量 p，操作方式与普通变量和数组没什么不同，只需要在引用时前面标明属于哪个结构体变量。

例 2：(2010-03-38)有以下定义和语句

```
struct  workers
{  int  num;
   char name[20];
   char c;
   srruct
   {int  day;int  month;int year;}  s;
};
struct workers  w,*pw;
pw=&w
```

能给 w 中 year 成员赋 1980 的语句是（ **D** ）

A）*pw.year=1980;　B）w.year=1980;

C）pw->year=1980;　D）w.s.year=1980;

【解析】引用 w 的内嵌结构体变量 s 的成员 year，应逐级递进。具体方法参考前面表格。

例 3：(2011-03-36)设有定义：struct {char mark[12];int num1;double num2;} t1,t2;，若变量均已正确赋初值，则以下语句中错误的是（C）

A)t1=t2;　　B)t2.num1=t1.num1;　　C)t2.mark=t1.mark;　　D)t2.num2=t1.num2;

【解析】结构体以及结构体成员的赋值，需要类型一致，所有选项都满足，但是 mark 是结构体中的字符数组的数组名，其数组的首地址，是一个指针常量，只能在初始化的时候赋值，故 C 错误。

例 4：(2011-03-37)有以下程序

```
struct ord
{ int x，y;}dt[2]={1,2,3,4};
main()
{
   struct ord *p=dt;
    printf("%d,",++(p->x)); printf("%d\n",++(p->y));
}
```

程序运行后的输出结果是(D)

A)1,2　　B)4,1　　C)3,4　　D)2,3

【解析】dt 是一个结构体类型的数组，其有两个结构体元素，p 是结构体类型指针，其指向 dt 即指向结构体数组的首地址，p->x 和 p->y 分别是结构体数组第一个元素的 x 成员和 y 成员的值，++在前则是先自增 1 再参与运算，所以输出的是 1+1=2，和 2+1=3，故选 D。

例 5：(2011-03-38)有以下程序

```
struct S
{ int a,b;}data[2]={10,100,20,200};
main()
{    struct S p=data[1];
     printf("%d\n",++(p.a));
}
```

程序运行后的输出结果是（D）

A)10　　B)11　　C)20　　D)21

【解析】结构体变量 p 赋值为结构体数组第二个元素的值，即 p.a=20,p.b=200;所以输出++(p.a)为 21，故选 D。

3.知识点：　结构体变量与函数调用

- 将一个结构体变量的值传递给另一个函数，有 3 个方法：

（1）用结构体变量的成员作参数。例如，用 stu[1].num 作实参，将值传递给形参。用法和用普通变量作实参是一样的，属于“**单向值传递**”方式。应当注意实参与形参的类型应保持一致。

（2）用结构体变量作实参。 对应形参也是同类型结构体变量，也属于“**单向值传递**”方式。

（3）用指向结构体变量（或数组）的指针作实参，将结构体变量（或数组元素）的地址传给形参。属于“**地址传递**”方式，应注意在被调用函数中是否对该地址的值有影响。

例 1：(2009-09-14)有以下程序运行后的输出结果是___16___

```
typedef  struct
{  int  num;double  s;}  REC;
void  fun1(REC  x)
{  x.num=23;x.s=88.5;  }
main()
{  REC  a={16,90.0};
    fun1(a);
    printf("%d\n",a.num);
}
```

【解析】main 函数中的 fun1(a); 是将 a 的值单向传递给了形参 x，相当于赋值给了 x，因此 x 的值的变化不会影响到 a

例 2：(2009-09-38)有以下程序

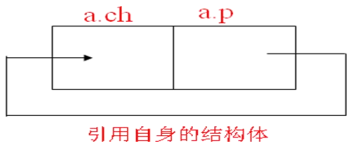
```
struct  A
{  int  a;char  b[10];double  c;};
struct  A  f(struct  A  t);
main()
{  struct  A  a={1001,"ZhangDa",1098.0};
    a=f(a);  printf("%d,%s,%6.1f\n",a.a,a.b,a.c);
```

}
struct A f(struct A t)
{ t.a=1002;strcpy(t.b,"ChangRong");t.c=1202.0;return t;}
程序运行后的输出结果是 （ **D** ）
A)1001,ZhangDa,1098.0
B)1002,ZhangDa,1202.0
C)1001,ChangRong,1098.0
D)1002,ChangRong,1202.0
【解析】注意在 f 函数中的 t.a=1002;strcpy(t.b,"ChangRong");t.c=1202.0; 这三条语句。

4.知识点：链表

- 当一个结构体中有一个或多个成员的基类型就是本结构体类型时,通常把这种结构体称为可以“**引用自身的结构体**”，也称为“**链表结构**”

例如：




```
struct link
{ char ch;
  Struct link *p;
}a;
```

在此，p 是一个**可以指向 struct link 类型变量**的指针成员，因此，**a.p=&a 是合法的表达式**，由此构成的存储结构如图所示。

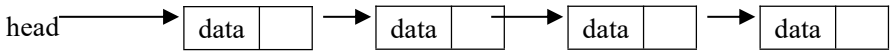
- **对链表进行的操作通常有以下四种：**

1.建立带有头结点的单向链表

2.顺序访问单向链表各结点数据域的值（即遍历链表）

```
struct std
{
    int data;   /*数据域*/
     struct std * next;   /*指针域*/
};
```

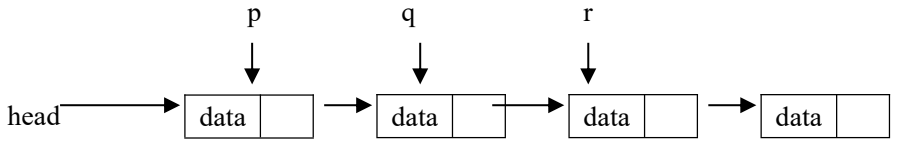
…建立头结点和数个节点，即建立以下链表



遍历链表：

```
p=head;
while(p!=0)
{
    printf("%4d",p->data);
    p=p->next;
}
}
```

3.删除单向链表中的某个结点

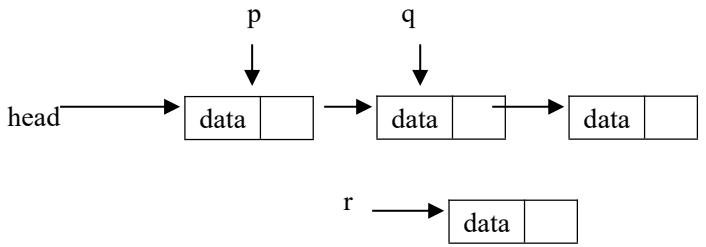


删除 q 指向的节点：

p->next=q->next;或者 p->next=r;

free(p);

4.向单向链表中插入结点



将 r 指向的节点插入 p、q 所指的两个结点之间：

r->next=q;　或者 r->next=p->next;

p->next=r;

例 1：(2011-09-37)在以下程序

```
#include<stdio.h>
main()
{struct node{int n; struct node *next;}*p;
 struct node x[3]={{2,x+1},{4,x+2},{6,NULL}};
p=x;
printf("%d,",p->n);
printf("%d\n",p->next->n);
}
```

程序运行后的输出结果是　（B）

A.2,3　　B.2,4　　C.3,4　　D.4,6

【解析】通过赋值语句 p=x，即 p=&x[0]，使 p 指向了 x[0]，因此 p->n 等价于 x[0].n，输出 2；p->next 等价于 x[0].next，即 x+1 或表示为&x[1]，p->next->n 也可表示为 x[1].n，输出 4

例 1：(2009-03-15)以下程序把三个 NODETYPE 型的变量链接成一个简单的链表，并在 while 循环中输出链表结点数据域中的数据，请填空

```
#include <stdio.h>
struct node
{int data; struct node *next;};
typedef  struct  node  NODETYPE;
main( )
{NODETYPE a,b,c,*h,*p;
a.data=10;b.data=20;c.data=30;h=&a;
b.next=&b;b.next=&c;c.next='\0';
p=h;
while(p){printf("%d",p->data);  __p=p->next__;}
}
```

【解析】while 就是用来遍历链表的，为了能遍历每个结点，每执行一次循环，就要使变量 p 指向下一个结点，所以空白处应填 p=p->next

5.知识点：共用体类型定义

- 共用体是一种构造数据类型（**遵循先定义类型，再定义该类型变量的原则**）
- 用途：使几个不同类型的变量**共占一段内存**(相互覆盖)
- 与结构体结构体与共用体

1.区别：存储方式不同，

结构体变量占用的内存单元,是其所有成员**占内存单元的总和**,而**共用体变量**是其所有成员中**占用空间最大的成员**的内存单元

2.联系：类型定义、变量定义、成员引用等操作方式相同，两种类型**可相互嵌套**

例 1.(06-04—41)有以下程序

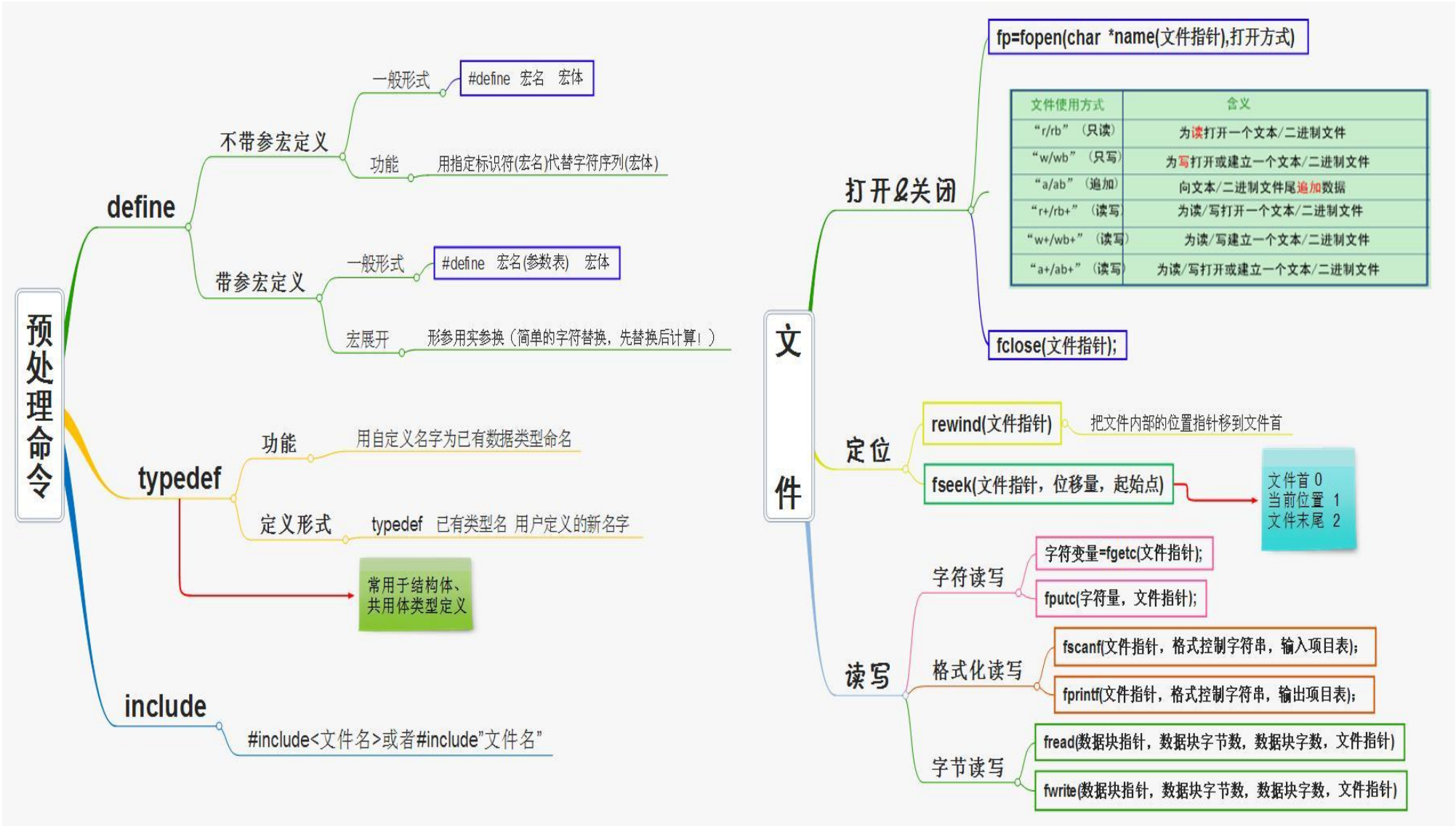
```
main()
{union
   {char ch [2];
      int d;
      }s;
   s.d=0x4321;
   printf("%x,%x\n", s.ch[0], s.ch[1]);
}
```

在 16 位编译系统上，程序执行后的输出结果是___A__

A)21，43　　B)43，21　　C)43，00　　D)21，00

【解析】此题考查整形数据在内存中存放形式以及共用体内存分配。对于共用体而言，字符数组 ch 与整型变量 d 共用 2 个字节的存储空间；存储器存储数据原则是：低地址存放低字节，高地址存放高字节；于是 0x4321 中 21 存放在低字节中，43 存放在高字节中，故输出时 s.ch[0]输出 21，s.ch[1]输出 43。

第六课 编译预处理、文件



本课重要知识点：

1.知识点：main()函数的参数

- 定义形式：main(int argc,char *argv[])
  {…
  }
- int argc 是命令行中的字符串数，char *argv[]是指向字符串的指针数组，系统使用空格把各个字符串隔开。

2.知识点：不带参数的宏定义

- 一般形式：#define 宏名 字符串（或数值）

即用字符串或数值取代宏名。

- 宏名一般采用大写字母表示，以与变量名区别，也可用小写字母。
- **宏定义不是 C 语句，不必在行末加分号**。

例 1：（2009-04-36）有以下程序

```
#include<stdio.h>
#define PT 3.5;
#define S(x) PT*x*x;
main()
{int a=l，b=2；printf("%4.1f\n"，S(a+b))；}
```

程序运行后输出的结果是（D）

A)14.0　　B)31.5　　C)7.5　　D)程序有错无输出结果

【解析】此题考查宏的定义形式。由于宏定义并不是语句，故#define 后面不能有分号，故该程序有错无法输出结果，所以，正确答案：D。

例 2：（2011-03-30）以下函数的功能是：通过键盘输入数据，为数组中的所有元素赋值。

```
#define N 10
void fun(int x[N])
{   int i=0;
    while(i<N) scanf("%d",_____);
}
```

在程序中下划线处应填入的是

A)x+i　　B)&x[i+1]　　C)x+(i++)　　D)&x[++i]

答案：C

【解析】要求给每个元素赋值，while 循环的变量 i 必须不断增加，所以 A、B 显然不行，i++是先参与运算在自增 1，++i 是先自增 1 再参与运算，所以 D 会出现 x[0]未赋值的情况，故选 C。

- 对程序中用“”括起来的字符串内的字符，即使与宏名相同，也不进行置换。
- 宏定义可以嵌套使用，例如：

```
#define R 3.0
#define PI 3.14159
#define L L*PI*R
```

3.知识点：带参数的宏定义

- 一般形式：#define 宏名（参数表） 字符串
- 带参数的宏不占运行时间，只占编译时间；只是简单、机械字符替换；宏名无类型。
- 要注意有括号和没括号的区别

例 1：（2009-09-35）有以下程序

```
#include <stdio.h>
#define f(x) x*x*x
main()
{int a=3,s,t;
s=f(a+1);      t=f((a+1));
printf("%d,%d\n",s,t);
```

}程序运行后的输出结果是（A）

A)10,64　　B)10,10　　C)64,10　　D)64,64

【解析】本题考点为带参数的宏定义，宏定义中的参数没有类型，仅为字符序列，不能当作表达式运算。宏展开时，把实参字符串原样写在替换文本中。

s=f(a+1)=a+1*a1*a+1=10；t=f((a+1))= (a+1)* (a+1)* (a+1)=64

例 2：(2011-03-35)有以下程序

```
#define S(x) 4*(x)*x+1
main()
{   int k=5,j=2;
    printf("%d\n",S(k+j));
}
```

程序运行后的输出结果是(B)

A)197　　B)143　　C)33　　D)28

【解析】#define 宏定义，只是机械简单的置换 S(k+j)= S(5+2)=4*(5+2)*5+2+1=143

4.知识点：文件包含

- 一般形式：#include <文件名>或者#include “文件名”
- <>与” ”的区别：<>即系统到存放 C 库函数头文件的目录中寻找要包含的文件；” ”即系统先在用户当前目录中寻找，若找不到，再到存放 C 库函数头文件的目录中寻找。
- 文件包含可以嵌套。
- 凡是以“#”开头的行都是预处理命令行。

5.知识点：calloc()函数

- 函数原型：void *calloc(unsigned n,unsigned size);
- 功能：分配 n 个数据项的内存连续空间，每个数据项的大小为 size

6.知识点：free()函数

- 函数原型：void free(void *p);
- 功能：释放 p 所指的内存区

7.知识点：malloc()函数

- 函数原型：void *malloc(unsigned size);
- 功能：分配 size 字节的存储区

例 1：(2011-03-14)以下程序运行后的输出结果是___emoclew___。

```
main()
{   char *p; int i;
    p=(char *)malloc(sizeof(char)*20);
    strcpy(p,"welcome");
    for(i=6;i>=0;i--) putchar(*(p+i));
    printf("\n"); free(p);
}
```

【解析】(char *)malloc(sizeof(char)*20)申请一个 20 个字符大小的空间，首地址赋给字符指针变量 p，strcpy(p,"welcome");将字符串"welcome"赋值给 p 所指向的空间，for 循环实现从后往前输出这个字符串，free(p)为释放申请的空间。

8.知识点：文件的概念和文件指针

- 在 C 程序中文件可以用顺序方式存取，可以用随机方式存取。
- C 程序把文件分为 ASCII 文件（文本文件）和二进制文件。

例 1：（2009-09-40）下列关于 C 语言文件的叙述中正确的是（C）

A)文件由一系列数据依次排列组成，只能构成二进制文件

B)文件由结构序列组成，可以构成二进制文件或文本文件

C)文件由数据序列组成，可以构成二进制文件或文本文件

D)文件由字符序列组成，其类型只能是文本文件

【解析】本题考点为文件的基本概念。正确答案为 C。C 语言中根据数据的组织形式，分为二进制文件和 ASCII 码文件。一个 C 文件时一个字节序列或者二进制序列，而不是一个记录（结构）序列。

- 文件类型指针：FILE *fp；fp 是一个指向 FILE 类型结构体的指针变量，通过文件指针变量找到与它相关的文件。

例 2：（2010-09-15）以下程序打开新文件 f.txt，并调用字符输出函数将 a 数组中的字符写入其中，请填空。

```
#include <stdio.h>
main()
{  FILE  *fp;
char a[5]={'1','2','3','4','5'},i;
fp=fopen("f.txt","w");
for(i=0;i<5;i++) fputc(a[i],fp);
fclose(fp);
}
```

9.知识点：fopen()函数和 fclose()函数

(1)fopen 函数

- 调用方式：fp=fopen("file1","r");
- 使用文件方式：如"r"只读打开一个文本文；"w"只写打开或建立一个文本文件；"a"追加打开一个文本文件，在文件末尾添加数据。

例 1：(2011-03-40)设 fp 已定义，执行语句 fp=fopen("file","w");后，以下针对文本文件 file 操作叙述的选项中正确的是(B)

A)写操作结束后可以从头开始读　　B)只能写不能读

C)可以在原有内容后追加写　　D)可以随意读和写

【解析】文件打开方式，'w'只能向该文件写入，如打开的文件不存在则以指定的文件名建立文件，如存在，则先删除再重建一个文件。故选 B。

- **若不能实现打开任务，fopen 函数将带回一个空指针值 NULL。**

例 1：（2009-04-13）以下程序用来判断指定文件是否能正常打开，请填空。

```
#include <stdio.h>
main()
{FILE *fp;
if(((fp=fopen("test.txt","r"))==  ____NULL_____))
printf("未能打开文件！\n);
else
printf("文件打开成功！\n");
}
```

【解析】考查文件打开操作。打开文件使用 fopen 函数，正常打开时返回指向该文件的文件指针，否则返回值为 NULL；题目中当 if 条件表达式成立时，输出"未能打开文件!"，故 fopen 函数返回值为 NULL。正确答案：NULL。

(2)fclose()函数

- 调用方式：fclose(fp);
- 顺利完成关闭操作，返回值为 0，否则返回 EOF（-1）。

10.知识点：fputc()函数和 fgetc()函数

- fgetc()函数的调用形式：ch=fgetc(fp);从打开的文件 fp 中读取一个字符。
- fputc()函数的调用形式：fputc(ch,fp);将字符输出到 fp 所指向的文件中去。

11.知识点：fread()函数和 fwrite()函数

- fread()函数的调用形式：fread(buffer,size,count,fp);其中 buffer 是一个指针，size 是每个数据块的字节数，count 是每次读入数据块的个数，fp 是文件指针。

例 1：struct stud

```
{char name[10];
 int num;
```

}stu[40];

假设学生的数据已存放在磁盘文件中，如何读入 40 个学生的数据？

for(i=0;i<40;i++)

fread(　　&stu[i]　　,　　sizeof(struct stur)　　,1,fp);

- fwrite()函数的调用形式：fread(buffer,size,count,fp);其中 buffer 是输出数据的首地址。

12.知识点：fscanf()函数和 fprintf()函数

- fscanf(文件指针，格式控制字符串，输入列表)；
- fprintf(文件指针，格式控制字符串，输出列表)；

例 1：（2011-09-40）有以下程序

#include<stdio.h>

main()

{FILE　*fp;

int　k,n,i,a[6]={1,2,3,4,5,6};

fp=fopen(“d2.dat”,”w”);

for(i=0;i<6;i++) fprintf(fp,”%d\n”,a[i]);

fclose(fp);

fp=fopen(“d2.dat”,”r”);

for(i=0;i<3;i++)　fscanf(fp,”%d%d”,&k,&n);

fclose(fp);

printf(“%d,%d\n”,k,n);

}程序运行后的输出结果是　　(C)

A.1,2　　B.3,4　　C.5,6　　D.123,456

【解析】第一次 fp=fopen(“d2.dat”,”w”);以写方式打开，通过循环将 a 数组全部写入文件中，并且每行写一个；第二次 fp=fopen(“d2.dat”,”r”); 以写方式打开，然后通过循环每次读取两个数字给 k 和 n，第三次 k 和 n 分别得到 5 和 6

例 1：（2009-04-40）有以下程序

#include <stdio.h>

main()

{FILE *f;

f=fopen(“filea.txt”,”w”);

fprintf(f,”abc”);

fclose(f);

}若文本文件 filea.txt 中原有内容为：hello，则运行以上程序后，文件 filea.txt 的内容为（C）

A)helloabc　　B)abclo　　C)abc　　D)abchello

【解析】程序中定义了一个文件指针，然后用 fopen()函数，以写的方式打开文件“filea.txt”，并让指针 f 指向它，执着调用 fprintf()函数，将“abc”写到指针所指向的文件中。由于 filea.txt 里原来的内容”hello”被”abc”覆盖掉，所以文件“filea.txt”里，最后的内容为”abc”，本题答案选 C。

13.知识点：fgets()函数和 fputs()函数

- fgets(str,n,fp);从 fp 指向的文件输入 n-1 个字符，在最后加一个’\0’，得到字符串共有 n 个字符，把它们放到字符数组 str 中。
- fputs(“China”,fp);把字符串”China”输出到 fp 指向的文件，字符串末尾’\0’不输出。

（2010-03-40）有以下程序

#include <stdio.h>

main()

{FILE *fp;char str[10];

fp=fopen(“myfile.dat”,”w”);

fputs(“abc”,fp);fclose(fp);

fp=fopen(“myfile,data”,”a+”);

fprintf(fp,”%d”,28);

rewind(fp);

fscanf(fp,”%s”,str);puts(str);

fclose(fp);

}程序运行后的输出结果是（C）

A)abc　　B)28c　　C)abc28　　D)因类型不一致而出错

【解析】以“a+”方式打开一个已存在的文件，原有文件内容不被删除，位置指针移到文件末尾，可作添加和读操作。fputs(“abc”,fp);把字符串”abc”输出到 fp 指向的文件。

14.知识点：rewind()函数

- 调用形式：rewind(fp);
- 功能：使位置指针重新返回文件的开头。

15.知识点：fseek()函数

- 功能：改变文件的位置指针
- 调用形式：fseek(fp,100L,0);将位置指针移到离文件头 100 个字节处。
- 文件首 SEEK_SET 0　　文件当前位置 SEEK_CUR 1
  文件末尾 SEEK_END 2

16.知识点：ftell()函数

- ftell(fp);
- 功能：得到流式文件中的当前位置。

17.知识点：feof()函数

- feof(fp);
- 判断文件是否结束，结束，返回值为 1，否则为 0。

例 1(2011-03-15)以下程序运行后的输出结果是 【15】 。

```
main()
{   FILE *fp; int x[6]={1,2,3,4,5,6},i;
    fp=fopen("test.dat","wb");
    fwrite(x,sizeof(int),3,fp);
    rewind(fp);
    fread(x,sizeof(int),3,fp);
    for(i=0;i<6;i++) printf("%d",x[i]);
    printf("\n");
    fclose(fp);
}
```

答案：123456

【解析】fwrite(x,sizeof(int),3,fp);将数组内的前三个元素即 1，2，3 输出到文件中去，rewind(fp);文件回到开头，fread(x,sizeof(int),3,fp);将文件的内容读出并存到 x 数组的前三个元素中，把 1，2，3 写到数组的前三个元素中去，所有数组中的元素值均没变。